

xv6 操作系统实验报告——MIT 6.S081 (2020) 全实验记录

2452292 · 付艺豪 · 2026

git 仓库: https://github.com/Fu-YH05/OS_xv6.git

本仓库以 MIT 6.S081 (Fall 2020) 课程实验为主线, 每个实验对应一个 git 分支, 代码可切换至不同分支查看。全部实验基于 RISC-V 版 xv6 教学操作系统, 覆盖用户态工具、内存管理、进程调度、文件系统、网卡驱动等内容。

- [xv6 操作系统实验报告——MIT 6.S081 \(2020\) 全实验记录](#)
 - [环境搭建](#)
 - [实验环境选择](#)
 - [安装 VMware 与 Ubuntu](#)
 - [安装工具链](#)
 - [获取实验代码](#)
 - [实验代码远程编写、组织与提交规范](#)
 - [Lab1 : Xv6 and Unix utilities](#)
 - [Boot xv6](#)
 - [Sleep](#)
 - [pingpong](#)
 - [primes](#)
 - [find](#)
 - [xargs](#)
 - [Lab2 : System calls](#)
 - [System call tracing](#)
 - [Sysinfo](#)
 - [Lab3 : Page tables](#)
 - [Print a page table](#)
 - [A kernel page table per process](#)
 - [Simplify copyin/copyinstr](#)

- [Lab4 : Traps](#)
 - [RISC-V assembly \(热身问答\)](#)
 - [Backtrace](#)
 - [Alarm](#)
- [Lab5 : Lazy_page allocation](#)
 - [Eliminate allocation from sbrk\(\)](#)
 - [Lazy allocation](#)
 - [Lazytests and Usertests](#)
- [Lab6 : Copy-on-Write fork](#)
- [Lab7 : Multithreading](#)
 - [Uthread: switching between threads](#)
 - [Using threads \(ph: 哈希表并行化\)](#)
 - [Barrier](#)
- [Lab8 : Locks](#)
 - [Memory allocator](#)
 - [Buffer cache](#)
- [Lab9 : File system](#)
 - [Large files](#)
 - [Symbolic links](#)
- [Lab10 : Mmap](#)
- [Lab11 : Network driver](#)

环境搭建

实验环境选择

xv6 实验需要一套 RISC-V 交叉编译工具链和 QEMU 模拟器。官方建议使用 Linux 环境，而我手头是一台 Windows 11 的机器。最开始我以为双系统是可以的，但是我对于Linux环境并不是很熟悉，所以我依赖VMware安装了Ubuntu20.04系统，渴望通过远程连接来实现编写代码和运行。

安装 VMware 与 Ubuntu

1. 打开VMware官网，下载软件的安装程序，我这里选用的是VMware Workstation PRO 17版本，下载好后直接双击进行安装即可。

2. 从网上下载 ubuntu-20.04.6-desktop-amd64.iso，下载成功后，启动VMware，并且点击创建新的虚拟机，选择ubuntu-20.04.6-desktop-amd64.iso，然后按部就班的进行虚拟机的创建。首次启动时设置 UNIX 用户名与密码。

安装工具链

进入 Ubuntu 后，先更新软件源，再一次性安装编译与调试所需的所有软件：

```
$ sudo apt-get update && sudo apt-get upgrade
$ git clone --recursive https://github.com/riscv/riscv-gnu-toolchain
$ sudo apt-get install autoconf automake autotools-dev curl libmpc-dev
libmpfr-dev libgmp-dev gawk build-essential bison flex texinfo gperf
libtool patchutils bc zlib1g-dev libexpat-dev
$ wget https://download.qemu.org/qemu-4.2.0.tar.xz
$ tar xf qemu-4.2.0.tar.xz
```

- `git clone --recursive https://github.com/riscv/riscv-gnu-toolchain`
`sudo apt-get install autoconf automake autotools-dev curl libmpc-dev libmpfr-dev libgmp-dev gawk build-essential bison flex texinfo gperf libtool patchutils bc zlib1g-dev libexpat-dev`：克隆 RISC-V GNU 编译器工具链的存储库，安装实验编译工具链所需要的包；
- `wget https://download.qemu.org/qemu-4.2.0.tar.xz`
`tar xf qemu-4.2.0.tar.xz`：下载 QEMU 4.2.0 的源代码，然后对 QEMU 4.2.0 的源代码包进行解压。

安装完成后验证版本：

```
$ qemu-system-riscv64 --version
```

如果能正常输出版本号，恭喜，工具链就绪！

提醒：实验2020版本支持的是 QEMU 4.x - QEMU 5.x 版本，如果下载最新版7.2.0版本，会导致无法正常启动内核。若遇到此问题，可退回旧版，或者将实验版本转为2021版本以及上。

获取实验代码

官方实验仓库通过 git 分支来区分不同实验：

```
$ git clone git://g.csail.mit.edu/xv6-labs-2020
$ cd xv6-labs-2020
$ git checkout util    # 以 util 实验为例
$ make qemu
```

编译完成后 QEMU 自动启动，看到如下画面就说明一切正常：

```
xv6 kernel is booting

hart 2 starting
hart 1 starting
init: starting sh
$
```

输入 `ls` 可以查看初始文件系统中预置的程序；按 `Ctrl-p` 可以打印进程信息；按 `Ctrl-a x` 退出 QEMU。

实验代码远程编写、组织与提交规范

先在虚拟机中，安装和启动ssh服务，然后得到虚拟机的ip。

我是通过VS中的Remote - SSH拓展和相应的edit拓展，借用远程资源管理器，输入了对应的ip，然后选择linux系统，建立连接后选择对应的实验代码文件夹（如果不额外设置的话每一次建立连接都要输入虚拟机的密码）。

提醒： 要注意的是，要留足Linux系统的磁盘内存，我最一开始没有留足空间，导致在进行远程连接的时候，要下载VS服务器，如果内存不足则无法正常下载，我后面通过磁盘拓展内存才得以正常进行连接。

每个实验的代码保存在独立分支中。实验的完整流程通常是：

1. `git checkout <分支名>` 切换到对应实验分支；
2. 修改代码，`make qemu` 编译运行；
3. 使用官方评分脚本进行测试，`make grade` 运行该实验全部测试；
4. 阶段性成果用 `git commit` 存档，出现问题可以随时回滚。
5. 最终成果通过 `git push destination util:util` 进行存档。

Lab1 : Xv6 and Unix utilities

本实验用于熟悉 xv6 及其系统调用。任务是在 xv6 中实现五个经典的 Unix 工具：sleep、pingpong、primes、find、xargs。

Boot xv6

切换到 util 分支并构建运行：

```
$ git checkout util
$ make qemu
```

看到 init: starting sh 提示符就说明 xv6 成功启动。输入 ls 可以查看初始文件系统，Ctrl-p 打印进程信息，Ctrl-a x 退出 QEMU。

Sleep

实验目的

实现 UNIX 程序 sleep，让进程暂停用户指定数量的时间片（tick）。tick 是 xv6 内核定义的时间单位，即定时器芯片两次中断之间的时间。程序放在 user/sleep.c；参数缺失或多余时要打印错误信息；main 最后要调用 exit() 退出。这个实验也用来走通"编写用户程序 → 修改 Makefile → 在 xv6 中运行"的流程，后面的实验都按这个流程来。

设计思路

内核已经实现了 sleep 系统调用（sys_sleep 在 kernel/sysproc.c 中），用户态只需要做三件事：

1. **参数检查**：argc != 2 时报错退出。不检查的话，无参数时 argv[1] 是空指针，atoi 会解引用崩溃。报错信息用 fprintf(2, ...) 写到 stderr，这样输出被重定向时错误提示仍然可见，参考 grep.c、rm.c 的做法。
2. **类型转换**：命令行参数是字符串（如 "10"），系统调用要整数，用 atoi（user/ulib.c）转换。atoi 不做合法性校验，非数字输入会得到 0。
3. **调用与退出**：调用 sleep(ticks) 后 exit(0)。xv6 的用户程序要显式调用 exit 做收尾，直接 return 的话进程状态会出问题。

实验内容

1. 创建 user/sleep.c，检查参数个数，参数错误时打印错误信息；
2. 用 atoi 将字符串参数转换为整数；

3. 调用 `sleep` 系统调用，最后 `exit()` 退出；

4. 在 Makefile 的 `UPROGS` 中加入 `$U/_sleep\`，`make qemu` 才会编译它。

```
#include "kernel/types.h"
#include "kernel/stat.h"
#include "user/user.h"

int main(int argc, char *argv[]) {
    // 检查参数数量，如果没有传入参数则报错提示
    if (argc != 2) {
        fprintf(2, "Usage: sleep <ticks>\n");
        exit(1);
    }

    // 将字符串参数转换为整型
    int ticks = atoi(argv[1]);

    // 调用系统调用 sleep
    sleep(ticks);

    exit(0);
}
```

遇到的问题与解决

错误信息的输出位置。 最初使用 `printf` 打印错误信息，后参照 `ls.c` 等官方程序改用

`fprintf(2, ...)`：文件描述符 2 是标准错误，错误信息与正常输出分流，管道拼接时不会把报错混进数据流。

实验心得

`sleep` 虽然简单，但完整展示了"用户程序 → 系统调用 → 内核"的调用链。`user/usys.pl` 会自动生成 `usys.S` 汇编桩，用户态的 `sleep()` 最终只是一条 `ecall` 指令，系统调用的本质是 CPU 的一条特殊指令。

pingpong

实验目的

用管道在两个进程之间传递一个字节，完成一次双向通信：父进程向子进程发送一个字节；子进程打印 `<pid>: received ping` 后把字节写回并退出；父进程读到字节后打印 `<pid>: received pong` 并退出。程序放在 `user/pingpong.c`。这个实验主要练习 `pipe`、`fork`、`read`、`write`、`getpid` 几个系统调用的配合使用。

设计思路

- 两条管道而非一条。** 管道是单向的（有独立的读端和写端）。如果只建一条管道并让父子同时读写，会出现“自己写的数据可能被自己读到”的混乱——子进程可能读到父进程写给它的字节，也可能读到它自己刚写回父进程的字节，无法区分。因此为每个方向各建一条管道：`p2c`（父→子）和 `c2p`（子→父）。
- 关闭不用的端口。** 这是本实验最关键的细节。`read` 只有在所有写端关闭后才返回 0（EOF）。如果子进程不关闭 `p2c` 的写端，父进程关闭自己那份写端后，管道仍有一个写端开着，父进程将永远等不到 EOF。反过来也一样。所以 `fork` 之后父子各自立即关闭自己不用的端口，这是正确性的必要条件，不是优化。
- 执行顺序的保证。** `read` 是阻塞操作：子进程的 `read` 等到父进程写完才返回，父进程的 `read` 等到子进程写回才返回。输出顺序（先 ping 后 pong）由管道语义保证，不需要额外的同步手段。
- 健壮性。** `fork` 失败时打印错误并退出；双方结束时关闭全部管道端口再 `exit`，避免文件描述符泄漏。

实验内容

- 创建两条管道并 `fork`；
- 父子双方各自关闭不用的管道端口；
- 父进程写入一个字节，子进程读到后打印并回写，父进程收到后打印。

```
#include "kernel/types.h"
#include "kernel/stat.h"
#include "user/user.h"

int main() {
    int p2c[2]; // 父进程到子进程的管道
    int c2p[2]; // 子进程到父进程的管道
    char buf[1];

    pipe(p2c);
    pipe(c2p);

    int pid = fork();

    if (pid < 0) {
        fprintf(2, "fork failed\n");
        exit(1);
    } else if (pid == 0) { // 子进程
        close(p2c[1]); // 关闭父到子管道的写端
        close(c2p[0]); // 关闭子到父管道的读端

        read(p2c[0], buf, 1);
        printf("%d: received ping\n", getpid());

        write(c2p[1], "x", 1);

        close(p2c[0]);
        close(c2p[1]);
        exit(0);
    } else { // 父进程
        close(p2c[0]); // 关闭父到子管道的读端
        close(c2p[1]); // 关闭子到父管道的写端

        write(p2c[1], "x", 1);

        read(c2p[0], buf, 1);
        printf("%d: received pong\n", getpid());
    }
}
```



```
        close(p2c[1]);
        close(c2p[0]);
        exit(0);
    }
}
```

遇到的问题与解决

父进程与子进程的同步。一开始担心父子执行的先后顺序，后来确认不需要额外处理：父进程的 `read` 阻塞到子进程写回为止，管道本身就完成了同步。子进程先打印 ping、父进程后打印 pong，输出顺序是确定的。

实验心得

pingpong 说明管道不仅是数据通道，还自带同步语义：读空管道会阻塞，写端全部关闭后 `read` 返回 0。一个字节的来回，背后是完整的进程间通信模型。

primes

实验目的

用管道和 fork 实现并发素数筛，思路来自 Unix 管道的发明者 Doug McIlroy：第一个进程把 235 写入管道；每遇到一个素数，就创建一个新进程，从左边管道读数据，把不能被这个素数整除的数写进右边的新管道。整个程序是一条进程流水线，最终打印 235 之间的全部素数。程序放在 `user/primes.c`。xv6 的文件描述符和进程数有限，不用的管道端口要及时关闭；主进程要等整个流水线（包括所有子孙进程）结束、所有输出打印完才能退出。

设计思路

- 1. 递归的筛子进程。** `sieve(pleft)` 刻画了一个筛子进程的行为：从左侧管道读出的第一个数必定是素数（前面每一层已把合数滤掉），打印；然后创建右侧管道并 fork，子进程递归成为下一层筛子，父进程继续从左侧读，把不能被当前素数整除的数写入右侧。每遇到一个素数，流水线就多出一段。
- 2. 终止条件。** 左侧管道 `read` 返回 0 时，说明上游所有写端已关闭、数据流结束，本层退出。每一层的父进程写完自己该传的数后关闭写端，EOF 信号逐层传播，整条流水线依次收尾。
- 3. 数据格式。** 直接写 4 字节的 `int`（`write(p[1], &i, sizeof(i))`）而不是格式化文本，读端按 `sizeof(p)` 对齐读，省去字符串解析。
- 4. 进程回收。** 每层父进程只 `wait` 自己的直接子进程，孙进程由下一层回收，链式回收保证 `main` 等到整条流水线结束才返回。

5. **资源管理**。每个进程进入 `sieve` 就关闭自己不用的写端；父进程 `fork` 后立即关闭右侧管道的读端，用完即关。

实验内容

1. `main` 创建第一条管道，把 2~35 全部写入，`fork` 出第一层筛选进程；
2. 每一层 `sieve` 从左侧管道读出的第一个数必定是素数，打印之；
3. 创建右侧管道并 `fork`：子进程递归进入下一层筛子，父进程继续读左侧管道，把不能被当前素数整除的数写入右侧管道；
4. 左侧 `read` 返回 0 时说明数据流结束，退出。

```

#include "kernel/types.h"
#include "kernel/stat.h"
#include "user/user.h"

void sieve(int pleft[2]) {
    int p;
    close(pleft[1]); // 关闭左侧管道的写端

    // 尝试读取第一个数（必定是当前过滤范围内的素数）
    if (read(pleft[0], &p, sizeof(p)) == 0) {
        close(pleft[0]);
        exit(0);
    }
    printf("prime %d\n", p);

    int pright[2];
    pipe(pright);

    int pid = fork();
    if (pid == 0) {
        // 子进程递归调用下一层筛子
        close(pleft[0]);
        sieve(pright);
    } else {
        close(pright[0]); // 父进程只往右侧写，关闭读端
        int n;
        while (read(pleft[0], &n, sizeof(n)) != 0) {
            if (n % p != 0) { // 过滤不能被 p 整除的数，传给下一个进程
                write(pright[1], &n, sizeof(n));
            }
        }
        close(pleft[0]);
        close(pright[1]); // 数据写完，关闭写端，等待子进程退出
        wait(0);
        exit(0);
    }
}

```

```

int main() {
    int p[2];
    pipe(p);

    int pid = fork();
    if (pid == 0) {
        sieve(p);
    } else {
        close(p[0]);
        for (int i = 2; i <= 35; i++) {
            write(p[1], &i, sizeof(i));
        }
        close(p[1]); // 全部写入后关闭写端
        wait(0);
        exit(0);
    }
    return 0;
}

```

遇到的问题与解决

文件描述符耗尽。 实验提示强调不用的文件描述符必须及时关闭，否则程序未算到 35 就会耗尽资源。每一层进程都继承着祖先的全部管道端口，放任不管的话描述符数量随层数线性膨胀。代码中每个进程进入 `sieve` 即关闭不用的端口，写完即关。

进程链的回收。 每层父进程只 `wait` 直接子进程，孙进程由下一层回收。链式回收保证流水线输出完毕后 `main` 才退出。

实验心得

每个筛子进程只做一件事：读入、判断、转发。没有共享内存，没有锁，靠管道的天然同步完成并发协作。

find

实验目的

实现简易版 UNIX `find`：递归查找目录树中指定名称的文件，打印完整路径。例如文件系统中存在 `./b` 和 `./a/b` 时，`find . b` 要输出这两行。程序放在 `user/find.c`。实现时注意三点：跳

过 `.` 和 `..` (否则无限递归); 字符串比较用 `strcmp` 而不是 `==`; 目录项名字是定长 14 字节, 填满时没有 `\0` 结尾。

设计思路

1. **以 `ls.c` 为模板。** `user/ls.c` 示范了"打开目录 → 读目录项 → 格式化文件名"的完整套路, `find` 的目录遍历部分基本是它的翻版。 `fmtname` 原样照搬: 从路径中找最后一个 `/`, 取其后部分作为文件名。
2. **递归结构。** `find(path, target)` 先 `open + fstat` 判断类型。普通文件比较名字, 匹配则打印; 目录把目录项名字拼接到 `path` 之后形成子路径, 递归调用 `find`。递归出口是遇到普通文件或目录读完。
3. **跳过 `.` 和 `..`。** 每个目录的前两个目录项是当前目录 `.` 和上级目录 `..`, 不跳过的话 `find` 会在 `.` 上无限自我递归。
4. **缓冲区与边界。** 子路径拼进 512 字节的 `buf` 前检查 `strlen(path) + 1 + DIRSIZ + 1 > sizeof(buf)`, 路径过长时打印错误并跳过。目录项名字用 `memmove` 拷满 14 字节后 `p[DIRSIZ] = 0` 强制补 `\0`, 避免 `strcmp` 读到名字之外的字节。
5. **错误处理。** `open / fstat` 失败、参数个数不对时打印错误到 `stderr` 并优雅退出, `find` 可能被用户指向不存在的路径。

实验内容

1. `fmtname` 从路径中提取最后一段文件名 (照搬 `ls.c`);
2. `find(path, target)` 先 `open + fstat` 判断类型: 普通文件比较名字; 目录拼接子路径后递归调用 `find`;
3. 遍历目录项时跳过 `.` 和 `..`, 防止无限递归;
4. 路径过长时检查缓冲区边界。

```

#include "kernel/types.h"
#include "kernel/stat.h"
#include "user/user.h"
#include "kernel/fs.h"

// 获取路径中的最后一部分文件名
char* fmtname(char *path) {
    static char buf[DIRSIZ+1];
    char *p;

    for(p=path+strlen(path); p >= path && *p != '/'; p--)
        ;
    p++;

    memmove(buf, p, strlen(p));
    buf[strlen(p)] = 0;
    return buf;
}

void find(char *path, char *target_name) {
    char buf[512], *p;
    int fd;
    struct dirent de;
    struct stat st;

    if((fd = open(path, 0)) < 0){
        fprintf(2, "find: cannot open %s\n", path);
        return;
    }

    if(fstat(fd, &st) < 0){
        fprintf(2, "find: cannot stat %s\n", path);
        close(fd);
        return;
    }

    switch(st.type){

```

```

case T_FILE:
    if (strcmp(fmtname(path), target_name) == 0) {
        printf("%s\n", path);
    }
    break;

case T_DIR:
    if(strlen(path) + 1 + DIRSIZ + 1 > sizeof buf){
        printf("find: path too long\n");
        break;
    }
    strcpy(buf, path);
    p = buf+strlen(buf);
    *p++ = '/';
    while(read(fd, &de, sizeof(de)) == sizeof(de)){
        if(de.inum == 0)
            continue;
        // 忽略 "." 和 ".." 以防死循环递归
        if(strcmp(de.name, ".") == 0 || strcmp(de.name, "..") == 0)
            continue;

        memmove(p, de.name, DIRSIZ);
        p[DIRSIZ] = 0;
        // 递归查找子目录或文件
        find(buf, target_name);
    }
    break;
}
close(fd);
}

int main(int argc, char *argv[]) {
    if(argc != 3){
        fprintf(2, "Usage: find <path> <filename>\n");
        exit(1);
    }
    find(argv[1], argv[2]);
}

```

```
exit(0);  
}
```

遇到的问题与解决

字符串比较。 C 语言里 `==` 比较的是指针而非内容，要用 `strcmp` 比较文件名，这是题目明确提示的陷阱。

目录项名字的结尾符。 xv6 的 `dirent.name` 是定长 14 字节数组，填满时没有 `\0`。用 `memcpy` 拷贝后再 `p[DIRSIZ] = 0` 强制补结尾符，避免 `strcmp` 越界读取。

实验心得

`find` 让我第一次上手 xv6 的文件系统接口：`open`、`fstat`、`read` 目录、`dirent` 结构。

xargs

实验目的

实现简易版 UNIX `xargs`：从标准输入逐行读取，把每一行作为参数追加到命令行指定的命令后面执行。例如 `echo hello too | xargs echo bye` 要输出 `bye hello too`。程序放在 `user/xargs.c`。UNIX 原版 `xargs` 会把多行合并成一次执行，本实验不要求这个优化，每行独立 `fork+exec` 一次即可。

设计思路

- 参数数组的构造。** `argv` 数组由两部分拼成——命令行里 `xargs` 自带的参数（如 `grep hello`）+ 从 `stdin` 读到的一行。先把自带参数依次拷入 `cmd_argv`，每读满一行就把缓冲区的地址追加到数组末尾，再补一个 `NULL` 结束符（`exec` 要求 `argv` 以 `NULL` 结尾）。数组大小用 `kernel/param.h` 的 `MAXARG`。
- 逐字节读行。** xv6 用户库没有 `fgets`，读行只能手动实现：循环 `read(0, &ch, 1)`，攒字符直到遇到 `\n`，把换行符替换成 `\0`。一行执行完毕后重置缓冲指针再读下一行。
- 每行一次 fork+exec+wait。** 子进程 `exec(cmd_argv[0], cmd_argv)` 执行命令；父进程 `wait(0)` 等子进程结束再处理下一行。不等待会导致多个子进程输出交错，而且子进程退出后成为僵尸进程堆积。`exec` 失败时打印错误并 `exit(1)`。
- 缓冲区复用。** 行缓冲 `buf[512]` 每行复用，上一行执行完必须重置指针；单行超长会越界（测试数据远小于 512 字节，属可接受的简化）。

实验内容

- 把命令行中 `xargs` 自带的参数预先拷入 `cmd_argv`；
- 从 `stdin` 逐字节读取，遇到 `\n` 补上 `\0` 后把整行作为最后一个参数；

3. `fork` 子进程执行 `exec(cmd_argv[0], cmd_argv)`，父进程 `wait` 等待执行完毕；
4. 重置缓冲指针，继续读下一行，直到 EOF。

```

#include "kernel/types.h"
#include "kernel/stat.h"
#include "user/user.h"
#include "kernel/param.h"

int main(int argc, char *argv[]) {
    if (argc < 2) {
        fprintf(2, "Usage: xargs <command> ...\n");
        exit(1);
    }

    char *cmd_argv[MAXARG];
    int cmd_argc = 0;

    // 复制命令行里自带的参数 (例如 xargs grep hello, 把 grep 和 hello 拷入)
    for (int i = 1; i < argc; i++) {
        cmd_argv[cmd_argc++] = argv[i];
    }

    char buf[512];
    char *p = buf;
    char ch;

    // 一次读取一个字符
    while (read(0, &ch, 1) == 1) {
        if (ch == '\n') {
            *p = '\0'; // 字符串结束符
            cmd_argv[cmd_argc] = buf; // 将新读到的一行作为最后一个参数
            cmd_argv[cmd_argc + 1] = 0; // 参数数组以 null 结尾

            if (fork() == 0) {
                exec(cmd_argv[0], cmd_argv);
                fprintf(2, "exec %s failed\n", cmd_argv[0]);
                exit(1);
            } else {
                wait(0); // 等待子进程执行完毕
            }
        }
    }
}

```

```
        // 重置指针准备读取下一行
        p = buf;
    } else {
        *p++ = ch;
    }
}

exit(0);
}
```

遇到的问题与解决

逐字节读而非逐行读。 xv6 的用户库没有 `fgets` 这类高层接口，只能逐字符 `read`，攒到换行符再处理，平时用惯的 `libc` 函数在这里需要自己实现。

wait 不可省略。 父进程不等待子进程，多个子进程的输出会交错混乱，且僵尸进程堆积。

实验心得

`xargs` 是 `fork + exec` 模式的典型应用：`exec` 替换当前进程的地址空间，所以必须在子进程中执行，父进程才能继续读下一行。`find`、`xargs`、`grep` 通过管道组合（`find . b | xargs grep hello`），体现了 Unix"小工具、大组合"的哲学。

Lab2 : System calls

本实验开始接触内核编程：为 xv6 添加两个系统调用——`trace`（系统调用追踪）和 `sysinfo`（系统信息收集）。添加系统调用要同时修改 `syscall.h`、`user.h`、`usys.pl`、`syscall.c` 等几个文件，这套流程在后面的实验里会反复用到。

System call tracing

实验目的

添加 `trace` 系统调用，接收一个整数参数 `mask`（掩码），每一位对应一个系统调用号（如追踪 `fork` 用 `trace(1 << SYS_fork)`）。内核在每个系统调用即将返回时，如果它的编号在掩码中置位，就打印一行：进程 ID、系统调用名、返回值。追踪只影响调用 `trace` 的进程及其 `fork` 出的子进程（包括 `exec` 之后的进程），不影响其他进程。

设计思路

1. **掩码的存放位置。** 在 `struct proc` 中新增 `trace_mask` 字段。追踪是每个进程的私有行为，随进程诞生（`fork` 继承）、随进程消亡（`proc` 释放时自然消失），打印时只需检查

`myproc()->trace_mask`，天然满足"只影响自己"的要求。

2. **打印时机**。在 `syscall()` 函数中、系统调用执行完毕之后打印。必须放在执行后的原因：输出格式要求包含返回值——只有等 `syscalls[num]()` 返回并把结果写入 `p->trapframe->a0` 之后，返回值才是已知的。判据用按位与：`(p->trace_mask & (1 << num)) != 0`。
3. **系统调用名字数组**。进程结构体的 `name` 字段是进程名，不是系统调用名。新建按系统调用号索引的字符串数组 `syscall_names[]`，用指定初始化器 `[SYS_fork] "fork"` 逐项填写，编号顺序变化时会直接编译报错。
4. **fork 继承**。在 `fork()` 中加一行 `np->trace_mask = p->trace_mask`。`exec` 不需要处理：`exec` 替换的是地址空间，进程结构体（含 `trace_mask`）原封不动——`trace mask cmd` 的工作方式正是设置掩码后 `exec` 成目标命令，掩码随进程延续。
5. **用户态程序**。`user/trace.c` 由课程提供，逻辑是先 `trace(atoi(argv[1]))` 再 `exec` 目标命令。

实验内容

添加系统调用需同步修改五个位置：

1. `kernel/syscall.h`：定义系统调用号；

```
#define SYS_trace 22
```

2. `user/user.h` + `user/usys.pl`：用户态函数声明与汇编桩生成（`usys.pl` 是 Perl 脚本，自动生成 `usys.S` 中的 `ecall` 桩代码）；
3. `kernel/proc.h`：在 `struct proc` 中新增掩码字段；

```
int trace_mask;           // 用于系统调用追踪的掩码
```

4. `kernel/sysproc.c`：实现 `sys_trace`，用 `argint` 从寄存器取出参数存入当前进程；

```
uint64
sys_trace(void)
{
    int mask;
    // 从寄存器 a0 获取传递的参数 (mask)
    if(argint(0, &mask) < 0)
        return -1;
    // 将 mask 保存到当前进程的结构体中
    myproc()->trace_mask = mask;
    return 0;
}
```

5. kernel/syscall.c: 把 `sys_trace` 注册进函数指针数组 `syscalls[]`; 新建按系统调用号索引的名字数组 `syscall_names[]`; 在 `syscall()` 中, 系统调用执行完毕、返回值写入 `a0` 之后检查掩码位并打印:

```
p->trapframe->a0 = syscalls[num](); // 执行系统调用并将返回值存入 a0

// 如果该系统调用对应的掩码位为 1, 则打印追踪信息
if ((p->trace_mask & (1 << num)) != 0) {
    printf("%d: syscall %s -> %d\n", p->pid, syscall_names[num], p-
>trapframe->a0);
}
```

6. kernel/proc.c 的 `fork()` 中继承掩码:

```
// 子进程继承父进程的 trace_mask
np->trace_mask = p->trace_mask;
```

7. Makefile 的 UPROGS 加入 `$U/_trace\`。

遇到的问题与解决

掩码继承的位置。 一开始把继承放在 `fork` 开头, 后来发现必须放在进程结构体复制完毕之后 (`np->cwd = idup(p->cwd)` 附近), 放在"还没复制完"的位置会导致子进程拿到错误的掩码值。

打印时机。 打印放在系统调用执行前则返回值尚未产生；放在执行后、`a0` 写入后才能连返回值一起输出。官方要求的输出格式决定了这个顺序。

实验心得

实现 trace 的过程完整展示了系统调用链路：`usys.pl` 生成汇编桩 → `ecall` 陷入内核 → `syscall()` 按 `a7` 寄存器中的编号查表分发 → 结果写回 `a0`。trace 本身也能被追踪（掩码含 trace 位时），这个自指现象说明系统调用机制的对称性。

Sysinfo

实验目的

添加 `sysinfo` 系统调用，接收一个指向 `struct sysinfo` 的用户空间指针，内核填充两个字段：`freemem`（空闲内存字节数）和 `nproc`（状态不为 `UNUSED` 的进程数）。结构体定义在 `kernel/sysinfo.h`。通过官方测试程序 `sysinfotest`（输出 `sysinfotest: OK`）即完成。

设计思路

- freemem 的统计。** 内存分配器 `kernel/kalloc.c` 用一个 `struct run` 单向链表（`kmem.freelist`）维护空闲物理页——链表每个节点就是一页（`PGSIZE` 字节）。统计空闲内存 = 持锁遍历链表计数 × `PGSIZE`。加锁的原因：多核环境下其他 CPU 可能正在并发分配/释放内存，不加锁遍历到一半链表被改，统计结果就会出错。
- nproc 的统计。** 进程表是 `kernel/proc.c` 中的全局数组 `proc[NPROC]`，每个进程有 `state` 字段。统计 = 遍历数组、逐个加锁检查 `state != UNUSED` 并计数。逐个加锁与内核其他代码的锁粒度保持一致，避免引入新的锁序依赖。
- 数据交付方式。** 在内核栈上构造 `struct sysinfo`，填好后用 `copyout(p->pagetable, addr, &info, sizeof(info))` 复制到用户指针指向的地址。`copyout` 会逐页检查用户地址合法性，非法地址返回 -1——这就是“坏指针测试”的拦截点。不能直接把内核栈地址返回给用户。
- 错误处理。** `argaddr` 取参数失败返回 -1；`copyout` 失败返回 -1。系统调用对用户传入的参数先验证后使用。

实验内容

与 trace 相同的注册流程（调用号 23、声明、桩、注册），核心实现分为三块：

- `kernel/kalloc.c` 中新增 `count_free_mem()`：加锁遍历空闲页链表，每块计 `PGSIZE` 字节：

```
// 计算系统剩余空闲内存（字节数）
uint64
count_free_mem(void)
{
    struct run *r;
    uint64 bytes = 0;

    acquire(&kmem.lock); // 必须加锁，保证并发安全
    r = kmem.freelist;
    while(r){
        bytes += PGSIZE;
        r = r->next;
    }
    release(&kmem.lock);

    return bytes;
}
```

2. `kernel/proc.c` 中新增 `count_process()`：遍历进程表，逐个加锁后检查 `state != UNUSED` 并计数：

```
// 计算状态不为 UNUSED 的进程数
uint64
count_process(void)
{
    struct proc *p;
    uint64 count = 0;

    for(p = proc; p < &proc[NPROC]; p++){
        acquire(&p->lock);
        if(p->state != UNUSED){
            count++;
        }
        release(&p->lock);
    }

    return count;
}
```

3. `kernel/sysproc.c` 中实现 `sys_sysinfo()`: `argaddr` 取用户指针 → 内核栈上构造结构体 → 填入统计结果 → `copyout` 拷贝到用户空间:


```

uint64
sys_sysinfo(void)
{
    uint64 st; // 用户空间的指针地址
    if(argaddr(0, &st) < 0)
        return -1;

    struct sysinfo info;
    info.freemem = count_free_mem();
    info.nproc = count_process();

    // 使用 copyout 将内核空间的数据安全地拷贝到用户空间指针指向的位置
    if(copyout(myproc()->pagetable, st, (char *)&info, sizeof(info)) <
0)
        return -1;

    return 0;
}

```

遇到的问题与解决

内核数据交付方式。 内核不能直接把内核栈上的结构体指针交给用户，会泄露内核地址。必须用 `copyout` 把数据复制到用户页表映射的地址中，这一个函数调用的背后是"用户指针不可信"的原则。

实验心得

`sysinfo` 的统计函数是第一次在内核里写"带锁遍历数据结构"的代码，锁是正确性的前提而不是可选项。系统调用对参数逐项校验、数据安全拷贝，这套习惯从这两个系统调用开始养成。

Lab3 : Page tables

本实验有三个任务：写一个打印页表的函数、为每个进程建立独立的内核页表、把 `copyin/copyinstr` 简化为直接解引用用户指针。

Print a page table

实验目的

编写 `vmprint()` 函数，按指定格式打印页表：第一行输出页表地址；之后每个有效 PTE 一行，按树的深度用 `..` 缩进（一级一个、二级两个、三级三个），每行输出 PTE 索引、PTE 原始值、提取出的物理地址，无效 PTE 不打印。在 `exec.c` 的 `return argc` 之前插入 `if(p->pid==1)` `vmprint(p->pagetable)`，打印第一个进程的页表。评分脚本会逐行正则匹配输出，缩进和十六进制宽度都要注意。

设计思路

1. **递归实现**。页表是天然的树：遍历当前页表页的 512 项 → 有效项打印 → 若是页目录节点则递归进入下一层。层数固定为 3，没有爆栈风险。
2. **页目录节点与叶子的区分**。这是本任务的核心判断。RISC-V 约定：指向下一级页表的 PTE 不设任何 R/W/X 权限位，而叶子 PTE 至少设置其中之一。于是 `(pte & (PTE_R|PTE_W|PTE_X)) == 0` 就是"继续递归"的判据，与 `freewalk` 释放页表时的判断一致。
3. **缩进实现**。递归函数带 `level` 参数（根为 1），打印前循环输出 `level` 组 `..`，中间用空格分隔。输出格式与官方示例一致，评分脚本用正则 `\s*\.\.\s*` 匹配缩进。
4. **打印格式**。用 `%p` 打印 PTE 与地址——xv6 的 `printf` 对 `%p` 输出定宽 16 位十六进制，匹配评分脚本的格式；索引用 `%d` 十进制。
5. **调用位置**。放在 `exec` 的 `return argc` 之前，此时新页表已装载完成。只打印 `pid==1`（init 进程），避免刷屏。

实验内容

```

// 辅助递归函数
void vmprint_helper(pagetable_t pagetable, int level) {
    for(int i = 0; i < 512; i++){
        pte_t pte = pagetable[i];
        // 只有当页表项有效时才打印
        if(pte & PTE_V){

            for(int j = 0; j < level; j++){
                if(j > 0) printf(" ");
                printf("..");
            }

            uint64 pa = PTE2PA(pte);

            printf("%d: pte %p pa %p\n", i, (void*)pte, (void*)pa);

            // 如果不是叶子节点，继续向下递归遍历
            if((pte & (PTE_R|PTE_W|PTE_X)) == 0){
                vmprint_helper((pagetable_t)pa, level + 1);
            }
        }
    }
}

// 打印主入口
void vmprint(pagetable_t pagetable) {
    printf("page table %p\n", (void*)pagetable);
    vmprint_helper(pagetable, 1);
}

```

调用位置在 `exec.c` 中 `proc_freepagetable(oldpagetable, oldsz)` 之后。此外在 `commit` 新页表之前也打印一次旧页表，用于对比 `exec` 前后的变化：旧页表只有 `trampoline`、`trapframe` 和一页 `initcode`，新页表则包含程序代码、数据和栈。

输出示例（第一行为页表地址，其余按深度缩进）：

```

page table 0x0000000087f6e000
..0: pte 0x0000000021fda801 pa 0x0000000087f6a000
.. ..0: pte 0x0000000021fda401 pa 0x0000000087f69000
.. .. .0: pte 0x0000000021fdac1f pa 0x0000000087f6b000
.. .. .1: pte 0x0000000021fda00f pa 0x0000000087f68000
.. .. .2: pte 0x0000000021fd9c1f pa 0x0000000087f67000
..255: pte 0x0000000021fdb401 pa 0x0000000087f6d000
.. ..511: pte 0x0000000021fdb001 pa 0x0000000087f6c000
.. .. .510: pte 0x0000000021fdd807 pa 0x0000000087f76000
.. .. .511: pte 0x0000000020001c0b pa 0x0000000080007000

```

对照课本图 3-4：第 0 页是进程的 text/data 页，第 2 页是用户栈，第 255 项映射 trampoline 与 trapframe。用户态不能读写第 1 页（guard page）：其 PTE 有效但 PTE_U 被清除。

遇到的问题与解决

%p 的格式。 grader 用正则匹配十六进制输出，xv6 的 printf 对 %p 打印定宽 16 位十六进制。若用 %x 打印 PTE 值，低位会被截断，定宽输出直接关系测试结果。

实验心得

vmprint 相当于给页表照 X 光，三级页表的每一级、每一项都能打印出来核对。这个函数在后面几个内存实验里也用来调试过。

A kernel page table per process

实验目的

为每个进程建立独立的内核页表。原来的 xv6 只有一个全局内核页表，不包含任何用户映射，内核拿到用户指针（比如 write 的缓冲区地址）时不能直接解引用，要靠 copyin/copyout 逐页翻译地址。本任务要求：给 struct proc 加内核页表字段；每个进程的内核页表包含内核映射、该进程的内核栈映射和用户映射；调度器切换进程时同步切换 satp，无进程运行时用全局内核页表；进程销毁时释放页表。两个注意点：用户映射进内核页表时要剥掉 PTE_U 位；用户地址空间不能越过 PLIC（0x0C000000）。

设计思路

1. 构造 proc_kpt_init()。仿照 kvmmake() 但返回新页表：kalloc 一页做根页表，逐项映射 UART0、VIRTIO0、PLIC、内核代码段（RX）、内核数据段（RW）、TRAMPOLINE。CLINT 不映射——xv6-2020 的时钟中断由 M 模式处理，S 模式从不访问 CLINT 寄存器，这是读 start.c 后得出的结论。

2. **内核栈迁移**。原来的 `procinit` 把所有进程的内核栈一次性映射进全局页表；改为在 `allocproc` 中 `kalloc` 内核栈物理页并映射进该进程自己的内核页表（虚拟地址仍按 `KSTACK(pid)` 布局）。
3. **satp 切换**。`scheduler()` 中选中进程后 `w_satp(MAKE_SATP(p->kpagetable)); sfence_vma();` 再 `swtch` ——必须在切换进程上下文之前换页表，因为新进程的内核栈只存在于它的内核页表里；`sfence_vma` 刷新 TLB 必不可少。进程切走后 `kvminithart()` 恢复全局内核页表。
4. **同步机制 `u2kvm_copy`**。用户页表每次变化（fork 复制、sbrk 增长、exec 装载），都要把 `[oldsz, newsz)` 区间的映射复制进内核页表。复制时 `flags & ~PTE_U` 剥离用户位——S 模式访问带 U 标志的页会触发硬件异常，这是本实验最容易出错的地方。
5. **释放**。`freeproc` 中先经 `walk(kpagetable, kstack)` 找到内核栈物理页并 `kfree`，再 `proc_free_kpt` 递归释放页表页。只释放页表自身的页（对叶子 PTE 跳过），用户物理页由用户页表的 `uvmfree` 负责，双重释放是内存系统的大忌。
6. **PLIC 保护**。`growproc` 与 `exec` 中检查地址空间上限 `>= PLIC` 即拒绝，否则用户地址涨进 PLIC 区间时，`u2kvm_copy` 会在内核页表中与 PLIC 映射冲突，触发 `mappages: remap panic`。

实验内容

1. **`proc_kpt_init()`** 仿照 `kvmmake` 为每个进程构造内核页表：

```
// 仿照 kvmmake() 映射内核运行所必需的硬件和代码段
// 注意：无需映射 CLINT（只有内核启动时需要）
mappages(kpt, UART0, PGSIZE, UART0, PTE_R | PTE_W);
mappages(kpt, VIRTIO0, PGSIZE, VIRTIO0, PTE_R | PTE_W);
mappages(kpt, PLIC, 0x400000, PLIC, PTE_R | PTE_W);
mappages(kpt, KERNBASE, (uint64)etext - KERNBASE, KERNBASE, PTE_R |
PTE_X);
mappages(kpt, (uint64)etext, PHYSTOP - (uint64)etext, (uint64)etext,
PTE_R | PTE_W);
mappages(kpt, TRAMPOLINE, PGSIZE, (uint64)trampoline, PTE_R | PTE_X);
```

2. **内核栈搬进 per-process 页表**：`procinit` 中原先集中分配内核栈的代码整段注释掉，改在 `allocproc` 中为每个进程 `kalloc` 内核栈并映射进自己的 `kpagetable`。
3. **`u2kvm_copy()`** 把用户页表 `[oldsz, newsz)` 区间的映射同步复制进内核页表，复制时剥离 `PTE_U` 位：

```
pa = PTE2PA(*pte);
// [关键] 必须剥离 PTE_U 权限位！
// 否则 RISC-V 处于 Supervisor 模式下时，硬件会拒绝访问带有 User 标记的内存。
flags = PTE_FLAGS(*pte) & (~PTE_U);
```

4. **调度器切换 satp**: `scheduler()` 中 `swtch` 之前 `w_satp(MAKE_SATP(p->kpagetable)); sfence_vma();`，进程切走后再 `kvminithart()` 恢复全局内核页表。切换页表后必须刷新 TLB，否则旧页表的缓存项仍在，CPU 会按旧页表翻译。
5. **释放**: `freeproc` 中先释放内核栈物理页，再 `proc_free_kpt` 递归释放页表自身。
6. **PLIC 上限保护**: `growproc` 和 `exec` 中分别加 `sz + n >= PLIC` 与 `sz1 >= PLIC` 的检查。

遇到的问题与解决

PTE_U 位导致的 page fault。第一次实现时原样复制了 flags，内核一碰用户页就 page fault。原因：RISC-V 特权级模型中，S 模式访问带 U 标志的页会触发异常，内核页表里的用户映射必须剥掉用户标志。

进程槽永久失效。调试时发现 `allocproc` 中 `trapframe` 分配失败路径忘了 `freeproc(p)`：泄漏内核栈和页表，且新进程状态设为 `USED` 后，该进程槽永远无法再分配。此问题在 `pgtbl` 分支遗留，后来在 `traps` 分支写 `alarm` 时发现并补上。

实验心得

每进程内核页表让内核能直接解引用用户指针，省去软件地址翻译的开销。这个设计后来因为 Meltdown/Spectre 一类侧信道攻击被 Linux 用 KPTI 替换，但在当时是很实用的优化。

Simplify copyin/copyinstr

实验目的

把 `copyin/copyinstr` 替换为对 `kernel/vmcopyin.c` 中 `copyin_new/copyinstr_new` 的调用。内核页表已经同步了用户映射，`copyin_new` 只需要先做地址边界检查，再一条 `memmove` 直接拷贝，地址翻译完全交给硬件。要保证用户页表每次变化（`fork`、`exec`、`sbrk`）时内核页表同步更新。

设计思路

1. **两行转发**。 `copyin` 主体变为 `return copyin_new(pagetable, dst, srcva, len);`，原实现整段注释保留作对照。真正的工程量在保证同步：`fork`、`exec`、`sbrk` 三处用户页表变化

点都已接上 `u2kvm_copy`，漏任何一处，内核页表看到的用户地址空间就是过期的。

2. **边界检查**（骨架预置）。`srcva >= p->sz`、`srcva+len >= p->sz`、`srcva+len < srcva`（溢出回绕）三重检查后执行 `memmove`。前两重检查挡不住"srcva 接近 MAXVA + 大 len"导致的和值溢出，第三重检查正是为此而生。
3. **统计验证**。`vmcopyin.c` 中的 `stats` 计数器与 `stats` 用户程序配合，可以对比新旧 `copyin` 的开销，grader 的 `test_count` 靠此计数器验证新路径被走到。

实验内容

核心改动只有两行转发，原来的逐页翻译代码整段注释保留：

```
int
copyin(pagetable_t pagetable, char *dst, uint64 srcva, uint64 len)
{
    /* ...原逐页软件拷贝代码全部注释保留... */
    return copyin_new(pagetable, dst, srcva, len);
}

int
copyinstr(pagetable_t pagetable, char *dst, uint64 srcva, uint64 max)
{
    /* ...原实现注释保留... */
    return copyinstr_new(pagetable, dst, srcva, max);
}
```

实验心得

把六十多行的软件地址翻译替换成一行 `memmove`，靠的是页表硬件替内核翻译地址。这也是 Linux 曾经的思路：让用户地址在内核中同样有效，省下海量的地址翻译开销。用硬件替代软件，是系统性能优化的第一选择。

Lab4 : Traps

陷阱（trap）是系统调用、中断和异常的共同入口。本实验先做 RISC-V 汇编热身，再实现栈回溯 `backtrace`，最后实现周期性的用户级定时器 `alarm`。

RISC-V assembly（热身问答）

实验目的

运行 `make fs.img` 编译 `user/call.c`，阅读生成的 `user/call.asm` 中 `g`、`f`、`main` 的汇编代码，回答六个问题。

实验内容

1. **哪些寄存器保存函数参数？** RISC-V 约定用 `a0 ~ a7` 传递前八个参数，更多参数走栈。`main` 调用 `printf("%d %d\n", f(8)+1, 13)` 时，13 存放在 `a2` 中（格式串在 `a0`，第一个参数在 `a1`）。
2. **main 中对 f 和 g 的调用在哪里？** 不存在。编译器在编译期算出 `f(8)+1 = 12`，把调用完全内联，汇编里只有一条 `li a1, 12`。
3. **printf 函数位于哪个地址？** 查阅 `call.asm`，`printf` 位于地址 `0x630`。
4. **main 中 jalr 调用 printf 之后，寄存器 ra 的值是什么？** `jalr` 把下一条指令的地址写入 `ra` 作为返回地址。`34: jalr 1536(ra)` 的下一条指令是 `38: li a0,0`，所以 `ra = 0x38`。
5. **运行 unsigned int i = 0x00646c72; printf("H%x Wo%s", 57616, &i); 输出什么？** 输出 `HE110 World`。57616 的十六进制是 `E110`；`i` 的四个字节按小端序存储为 `72 6c 64 00`，对应 ASCII `"rld\0"`——`H + E110 + Wo + rld = "HE110 World"`。若 RISC-V 是大端序，需把 `i` 改为 `0x726c6400`；57616 是数值而非字符串，无需改变。
6. **printf("x=%d y=%d", 3); 的 y= 之后会打印什么？** 一个不确定的值。格式串声明了两个参数，调用方只传了一个，`printf` 从寄存器中读取"幽灵"第二参数，具体值取决于当时的寄存器内容。这是典型的未定义行为。

实验心得

六个问题覆盖 ABI 约定、编译优化、链接布局、大小端和未定义行为。"看汇编"是系统程序员的基本功，内核调试中 `kernel.asm` 是比源码更可靠的依据。

Backtrace

实验目的

在 `kernel/printf.c` 中实现 `backtrace()`：利用栈帧指针（`s0` 寄存器）向上遍历调用栈，打印每层栈帧的返回地址。在 `sys_sleep` 中插入调用（测试程序 `bttest` 只调用 `sleep(1)`），并在 `panic()` 中调用，让内核崩溃时自动打印回溯。栈帧布局：`fp-8` 处是返回地址，`fp-16` 处是上一帧的帧指针；`xv6` 的每个内核栈恰好一页，用 `PGROUNDOWN(fp) / PGROUNDUP(fp)` 圈出栈页范围作为终止边界。

设计思路

1. **读帧指针。** `kernel/riscv.h` 中用内联汇编读 `s0`：`asm volatile("mv %0, s0" : "=r"(x))`。RISC-V 没有"读 `s0`"的专用 CSR，内联汇编是唯一途径。选用 `s0` 的原因：编译器在 -

`fno-omit-frame-pointer` (xv6 Makefile 已设置) 下把 `s0` 专用于保存帧指针，这是 `backtrace` 能工作的前提。

2. **回溯循环。** 从当前 `fp` 出发，打印 `*(uint64*)(fp-8)` (返回地址)，再 `fp = *(uint64*)(fp-16)` 取上一帧，循环直到 `fp` 超出当前栈页范围。终止条件的选择是重点：只设上界 (`fp < PGR0UNDUP(fp)` 初始值) 依赖"内核栈帧链上的 `fp` 单调递增 (栈向下生长)，追到用户态帧或垃圾值时必然大于栈页上界"；严格做法是同时检查下界 `fp >= PGR0UNDDOWN(fp)`，防止追到小值时继续向下读触发内核缺页。本实现只检查上界，依赖 `kalloc` 对空闲页的 `0x05` 填充充当大垃圾值兜底，测试可通过，但双边界检查更稳妥，详见"遇到的问题"。
3. **调用点。** 官方提示在 `sys_sleep` 中临时插入调用以便测试。评分脚本 `bttest` 是只调 `sleep` 的小程序，把调用保留在 `sys_sleep` 中是必要的；`panic()` 中的调用让之后所有内核崩溃都自带回溯。

实验内容

1. 在 `kernel/riscv.h` 中用内联汇编读取 `s0`：

```
static inline uint64
r_fp()
{
    uint64 x;
    asm volatile("mv %0, s0" : "=r" (x) );
    return x;
}
```

2. **遍历栈帧：** `fp-8` 处是返回地址，`fp-16` 处是上一层帧指针，循环沿帧链向上走直到走出当前栈页：

```
// [新增] 打印调用栈返回地址
void
backtrace(void)
{
    printf("backtrace:\n");

    uint64 fp = r_fp();
    uint64 bottom = PGROUNDUP(fp); // 栈底（高地址边界）

    // 遍历调用栈，直到超出当前 4KB 栈页面
    while (fp < bottom) {
        uint64 ra = *(uint64*)(fp - 8); // 返回地址在 fp - 8
        printf("%p\n", ra);
        fp = *(uint64*)(fp - 16);      // 上一个 fp 在 fp - 16
    }
}
```

3. 在 `panic()` 中调用 `backtrace()`；同时在 `sys_sleep` 末尾加一次调用，评分测试 `bttest` 是一个只调用 `sleep(1)` 的小程序，没有这次调用回溯不会出现。

`bttest` 打印的 3 条地址经 `addr2line -e kernel/kernel` 翻译后落在 `kernel/sysproc.c`、`kernel/syscall.c`、`kernel/trap.c`——即 `sys_sleep` → `syscall` → `usertrap` 的调用链。

遇到的问题与解决

循环终止条件。 官方提示用 `PGROUNDUP(fp)` 和 `PGROUNDUP(fp)` 圈出当前栈页范围，作为循环的上下界。本实现只检查上界：内核栈帧的 `fp` 依次递增，追到用户态帧指针（远低于内核栈页）时必然大于上界。实际终止依赖 `xv6` 的 `kalloc` 把空闲页填充成 `0x0505050505050505`，用户栈残留的垃圾帧指针恰好是这种大值。严格来说同时检查上下界才是稳妥写法——只靠垃圾值兜底属于“碰巧正确”，若用户程序把自己的 `s0` 清零，回溯会一路读向地址 0 触发内核缺页。

实验心得

`backtrace` 说明栈帧是编译器和硬件共同写下的记录：没有调试器、没有符号表，仅凭一个寄存器里的指针就能重建调用历史。之后内核 `panic` 时能直接看到调用栈，定位问题方便了很多。

Alarm

实验目的

添加 `sigalarm(interval, handler)` 与 `sigreturn()` 两个系统调用：进程每消耗 `interval` 个 tick 的 CPU 时间（只统计该进程实际占用的时间），内核就安排它执行一次用户函数 `handler`；`handler` 调用 `sigreturn()` 后，进程从被打断的指令处继续执行，所有寄存器恢复原样；`sigalarm(0, 0)` 取消定时警报。`alarmtest` 有三个测试：`test0` 要求 `handler` 能被调用；`test1` 要求寄存器完整恢复；`test2` 要求防重入（`handler` 没返回时不能再次触发）。

设计思路

- 1. 进程状态五字段。** `alarm_interval`（间隔，0 表示禁用）、`alarm_handler`（用户函数地址）、`alarm_ticks`（距上次触发的累计 tick）、`is_alariming`（防重入标志）、`alarm_trapframe`（独立 `kalloc` 一整页，保存被打断瞬间的全部用户寄存器）。单独分配一页而非在 `trapframe` 页内借空间，语义清晰、无越界风险，代价是每进程多一页内存。
- 2. 触发机制。** `usertrap` 中 `which_dev == 2` 分支（时钟中断）里，若 `interval != 0` 且不在报警中，`alarm_ticks++`；计数达到间隔时：备份 `trapframe` 到 `alarm_trapframe` → 置 `is_alariming = 1` → 清零计数 → 把 `p->trapframe->epc` 改成 `handler` 地址。不在内核态直接调用 `handler`：内核页表无法执行用户函数，权限也不对。正确做法是“借道返回”——`usertrapret` 返回用户态时 CPU 从 `epc` 恢复执行，`epc` 指向 `handler`，即自然跳入。
- 3. 恢复机制。** `sys_sigreturn` 把 `alarm_trapframe` 整体 `memmove` 回 `trapframe`（`epc`、`sp`、全部通用寄存器一次复原），清 `is_alariming` 标志。返回值必须是 `p->trapframe->a0`：系统调用返回值写入 `a0`，但 `memmove` 已把用户程序的原始 `a0` 恢复，不显式返回它用用户程序的 `a0` 就被覆盖。
- 4. 防重入（冻结计时方案）。** `is_alariming == 1` 期间连 `alarm_ticks++` 都不执行，计时完全冻结；`sigreturn` 清标志后从头累计。这样 `handler` 运行期间不可能重入，两次触发之间至少间隔一个完整 `interval`，`test2` 的 `slow_handler` 场景下天然通过。
- 5. 注册流程。** 与 Lab2 同构的五处接线（编号 22/23、声明、`usys.pl`、`syscall.c` 表、`Makefile` 加 `alarmtest`）。

实验内容

- 1. struct proc 新增五个字段：**

```
// 报警器相关属性
int alarm_interval;           // 报警间隔 tick 数 (0 表示禁用)
void (*alarm_handler)();     // 用户空间的报警处理函数地址
int alarm_ticks;              // 自上次报警后累积经过的 tick 数
int is_alariming;             // 防重入标志位 (1 表示正在处理报警)
struct trapframe *alarm_trapframe; // 专门保存报警打断瞬间的用户寄存器状态
```

2. `usertrap` 中处理时钟中断：计数递增，达到间隔时备份 `trapframe`、把 `epc` 篡改为 `handler` 地址：

```
// [新增] 处理时钟警报逻辑
if(p->alarm_interval != 0 && p->is_alarming == 0) {
    p->alarm_ticks++;
    if(p->alarm_ticks >= p->alarm_interval) {
        // 触发报警
        p->is_alarming = 1;
        p->alarm_ticks = 0;

        // 备份当前的陷入帧（里面有所有的用户态寄存器和原本马上要返回执行的 PC
        (epc))
        memmove(p->alarm_trapframe, p->trapframe, sizeof(struct
        trapframe));

        // 篡改返回地址：将 PC 指向用户设定的 handler。
        // 这样 usertrap 返回用户态时，就会跳转去执行 handler 而不是原来执行到一半
        的代码
        p->trapframe->epc = (uint64)p->alarm_handler;
    }
}
```

3. `sys_sigreturn` 恢复现场并解除防重入标志：

```

uint64
sys_sigreturn(void)
{
    struct proc *p = myproc();

    // 恢复时钟中断打断瞬间的所有寄存器现场
    memmove(p->trapframe, p->alarm_trapframe, sizeof(struct
trapframe));

    // 退出报警处理状态，允许下次报警重新触发
    p->is_alarmed = 0;

    // 返回 a0 寄存器里的原始值（因为上面 memmove 覆盖了，系统调用本身返回值也是放
a0，不写这句原程序的 a0 会被破坏）
    return p->trapframe->a0;
}

```

注意最后一句 `return p->trapframe->a0`： `syscall()` 会把系统调用返回值写入 `a0`，但 `memmove` 已经把被中断程序的原始 `a0` 恢复出来了——如果不显式返回它，用户程序的 `a0` 寄存器就被返回值覆盖了。

- 注册流程与 Lab2 同构： `syscall.h` 编号、 `usys.pl` 桩、 `syscall.c` 数组、 `user.h` 声明、 `Makefile` 加入 `$U/_alarmtest`。

遇到的问题与解决

用户态 handler 的"调用"方式。 在内核里直接跳转到用户地址不可行：页表还是内核页表，权限也不对。正确思路是借 `usertrapret` 的返回路径——把 `trapframe->epc` 改成 handler 地址，返回时 CPU 自然跳进 handler。

a0 被覆盖。 第一版 `sys_sigreturn` 直接 `return 0`， `alarmtest` 的 `test1` 偶发失败。排查发现：恢复现场后，用户程序原本的 `a0` 被系统调用返回值 0 覆盖，而用户程序的循环变量存在寄存器里， `a0` 一错程序逻辑全乱。改用 `return p->trapframe->a0` 把原始值传回后问题解决。

实验心得

`alarm` 是第一次实现用户级中断处理：备份现场 → 改返回地址 → 用户态执行 handler → `sigreturn` 恢复现场，和 Linux 的信号处理是同一个思路。

Lab5 : Lazy page allocation

实验目的

把 xv6 的 `sbrk()` 改成惰性分配。原来的 `sbrk` 会立即为申请的每一页分配物理内存并建立映射，1GB 请求要分配 26 万个物理页，既慢又浪费——很多程序申请了内存只用一小部分（稀疏数组），或者提前申请、很久之后才用。改造后：`sys_sbrk` 只修改 `p->sz`（`n>0` 增加；`n<0` 用 `uvmdealloc` 立即回收），不分配物理内存；进程第一次访问这些地址时触发缺页（`scause` 13/15），内核在 `usertrap` 中捕获，合法地址就分配一页、清零、映射，非法地址（超过 `sz` 或碰到栈下 `guard page`）就杀死进程。`lazyttests` 和 `usertests` 还考察这些边界：负参数 `sbrk`；OOM 时杀进程而非崩溃；`fork` 正确处理"有空洞"的地址空间；把未触碰的 `sbrk` 地址传给 `read/write/pipe` 也要能工作。

设计思路

1. **分离"记账"与"兑现"。** `p->sz` 是记账（承诺给用户的虚拟内存范围），页表映射是兑现（实际可用的物理内存）。惰性分配把两者解耦：`sbrk` 只改 `sz`，兑现推迟到缺页。内核里凡是假设"sz 范围内必有映射"的代码都要重新审视——`uvmunmap`（`exit` 回收）、`uvmcopy`（`fork` 复制）首当其冲。
2. **缺页处理的分诊逻辑。** `usertrap` 中按 `r_stval()` 拿到缺页地址后分三类：`va >= p->sz` 或 `va < PGROUNDDOWN(sp)`（栈下）为非法，杀进程；否则为合法懒页，`kalloc` + `memset` 清零 + `mappages` 映射（权限 `RWXU`，与堆一致）。`kalloc` 或 `mappages` 失败同样杀进程。
3. **放宽 panic 的两处。** `uvmunmap` 对 `walk` 失败/PTE 无效的页由 `panic` 改为 `continue`（退出时回收"有空洞"的地址空间）；`uvmcopy` 同样 `continue`（`fork` 继承父进程的空洞）。
4. **内核侧访问懒页——walkaddr 补分配。** 修订版 `usertests` 的 `sbrkarg` 测试要求把刚 `sbrk` 出来、从未触碰的地址传给 `write(fd, a, PGSIZE)`、`pipe((int*)a)` 也必须成功。此时访问用户地址的是内核（`copyout/copyin`），走的是 `walkaddr` 软件翻译而不是缺页处理。因此在 `walkaddr` 中加补分配逻辑：PTE 不存在 → 边界检查 → `kalloc`/清零/`mappages` → 返回新物理页地址。所有内核地址翻译路径由此自动获得"懒页即补"的能力。
5. **负 sbrk 的语义。** `p->sz = uvmdealloc(pagetable, sz, sz+n)` 立即回收——缩小的空间里已分配的页被解除映射并释放，未分配的直接跳过。

实验内容

Eliminate allocation from sbrk()

本步骤对应官方第一步（easy）：移除 `sbrk` 中的物理内存分配。修改 `kernel/sysproc.c` 的 `sys_sbrk`：`n` 为正时只把 `p->sz` 加上 `n`；`n` 为负时用 `uvmdealloc` 立即回收。做完这步 `echo hi` 都会崩溃——`shell` 写栈上方新地址时缺页，内核不认识这个异常。

```

uint64
sys_sbrk(void)
{
    int addr;
    int n;
    struct proc *p = myproc();

    if(argint(0, &n) < 0)
        return -1;

    addr = p->sz;

    // 处理负数的情况：立刻回收多余的物理页面
    if (n > 0) {
        p->sz += n;
    } else if (n < 0) {
        p->sz = uvmdealloc(p->pagetable, p->sz, p->sz + n);
    }

    return addr;
}

```

Lazy allocation

本步骤对应官方第二步（moderate）：在 `usertrap` 中处理缺页。 `r_scause()` 为 13（读缺页）或 15（写缺页）时进入懒分配逻辑：

```

} else if(r_scause() == 13 || r_scause() == 15) {
    // [新增] 处理 Page Fault (Lazy Allocation)
    uint64 va = r_stval(); // 获取发生缺页的虚拟地址

    // 边界检查：
    // 1. va >= p->sz：地址超出了进程申请的最大堆内存
    // 2. va < p->trapframe->sp：地址低于用户栈底（极大概率是碰到了 guard page
    // 无效页）
    if(va >= p->sz || va < PGROUNDDOWN(p->trapframe->sp)) {
        p->killed = 1; // 违规访问，直接杀死进程
    } else {
        // 申请一个物理页
        char *mem = kalloc();
        if(mem == 0) {
            // OOM (Out of Memory) 内存耗尽
            p->killed = 1;
        } else {
            memset(mem, 0, PGSIZE);
            // 将新申请的物理页映射到发生缺页的虚拟地址上
            if(mappages(p->pagetable, PGROUNDDOWN(va), PGSIZE, (uint64)mem,
PTE_W|PTE_X|PTE_R|PTE_U) != 0){
                kfree(mem); // 映射失败，释放内存并杀死进程
                p->killed = 1;
            }
        }
    }
} else {

```

两个边界检查缺一不可：地址超过 `sz` 说明进程在访问从未申请过的内存，必须杀；地址低于栈底说明进程触碰了栈下方的 guard page，同样必须杀。否则任意非法访问都会被分配一页，安全边界形同虚设。

Lazytests and Usertests

本步骤对应官方第三步（moderate）：处理 lazytests 与 usertests 考察的各类边界情况。

uvmunmap 与 uvmcopy 放宽（对应 lazy unmap 与 fork 场景）：

- `uvmunmap`： `walk` 失败或 PTE 无效时由 `panic` 改为 `continue` ——进程退出时要回收一个散布着大量未分配页的地址空间，跳过才是正解。
- `uvmcopy`（fork 时复制）：同样对无效 PTE `continue` ——子进程继承父进程的"空洞"地址空间。

walkaddr 补分配（对应 `usertests` 的 `sbrkarg` 场景）：官方标准解法只处理用户态缺页，但修订版 `usertests` 里的 `sbrkarg` 测试要求：把刚 `sbrk` 出来、从未触碰过的地址传给 `write(fd, a, PGSIZE)` 甚至 `pipe((int*)a)` 也必须成功。此时是内核在访问用户地址（`copyout/copyin`），走的是 `walkaddr` 而不是缺页处理。因此在 `walkaddr` 中加了补分配逻辑：

```

uint64
walkaddr(pagetable_t pagetable, uint64 va)
{
    pte_t *pte;
    uint64 pa;

    if(va >= MAXVA)
        return 0;

    pte = walk(pagetable, va, 0);

    // [新增] 如果 PTE 不存在, 或者 PTE 无效, 说明可能是一个尚未分配的 Lazy Page
    if(pte == 0 || (*pte & PTE_V) == 0){
        struct proc *p = myproc();

        // 检查越界和 guard page
        if(va >= p->sz || va < PGROUNDDOWN(p->trapframe->sp))
            return 0;

        // 尝试立刻为这个系统调用补齐物理内存分配
        char *mem = kalloc();
        if(mem == 0)
            return 0;
        memset(mem, 0, PGSIZE);

        if(mappages(p->pagetable, PGROUNDDOWN(va), PGSIZE, (uint64)mem,
PTE_W|PTE_X|PTE_R|PTE_U) != 0){
            kfree(mem);
            return 0;
        }
        // 分配成功, 直接返回这块新的物理页地址
        return (uint64)mem;
    }

    if((*pte & PTE_U) == 0)
        return 0;
    pa = PTE2PA(*pte);

```

```
return pa;
}
```

内核访问到"懒页"就立刻补齐，缺页处理对内核路径同样生效。

遇到的问题与解决

sbrkarg 测试无法通过。 用户态缺页处理没问题，但 `write(fd, a, PGSIZE)` 里 `a` 是未触碰的 `sbrk` 地址——内核 `copyout` 翻译地址时发现 PTE 无效，直接返回失败。该测试要求内核侧也支持懒分配，把补分配逻辑放进 `walkaddr`（所有内核地址翻译的必经之路）后问题解决。

进程退出时的海量未映射页。 `sbrk(1GB)` 的进程被杀后，`exit` 路径的 `uvmunmap` 要遍历 26 万个页表项，其中绝大多数从未分配，原版代码遇到第一个"not mapped"就 panic。改成 `continue` 后退出路径畅通。

实验心得

惰性分配的核心是"记账"和"兑现"分离：`sbrk` 返回的只是承诺，物理内存推迟到真正访问时再给。很多程序申请 1GB 只用几 MB，这样能省下大量不必要的分配。

Lab6 : Copy-on-Write fork

实验目的

把 `fork()` 改成写时复制 (COW)。原来的 `fork` 会把父进程的全部用户内存逐页复制给子进程，内存大时很慢，而且大部分复制是浪费的——`fork` 后紧跟 `exec` 的常见模式里，复制的内容几乎全被丢弃。COW `fork` 的做法：子进程页表直接指向父进程的物理页，双方 PTE 都去掉写权限、打上 COW 标记；谁写谁触发缺页，内核再分配新页、复制内容、恢复写权限。物理页要维护引用计数：`kalloc` 分配时置 1，`fork` 共享时加 1，解除映射时减 1，归零才真正放回空闲链表。`cowtest` 和 `usertests` 都要通过。

设计思路

- COW 标记的来源。** RISC-V PTE 的第 8、9 位是保留给操作系统软件使用 (RSW) 的位，硬件完全忽略——`#define PTE_COW (1L << 8)` 完美借位，零硬件成本。
- 引用计数的数据结构。** 全局数组 `int count[PHYSTOP / PGSIZE]`，索引 = 物理地址 / PGSIZE，配一把独立的自旋锁。初始化技巧：`kinit` 先把所有页计数置 1，再让 `freerange` → `kfree` 逐个减到 0 入 `freelist`——任何 `kfree` 都不会把计数减成负数。`kalloc` 分配新页时计数重置为 1；`kfree` 先减计数，大于 0 直接返回（共享页），等于 0 才真正释放，负数 panic。

3. **uvmcopy 的改造**。不 `kalloc`、不 `memmove`，只做三件事——可写页剥夺 `PTE_W`、加上 `PTE_COW`（父进程 `PTE` 原地修改，从此父子写它都会缺页）；子进程 `mappages` 指向同一物理页；`cow_ref_add` 计数加一。只读页（如代码段）不需要 `COW` 标记，但同样计数加一（被两个页表共享）。
4. **缺页处理 `cow_page_fault`**。四重检查（`MAXVA` 越界、`PTE_V`、`PTE_U`、`PTE_COW`）确认是合法 `COW` 页后：`kalloc` → `memmove` 拷贝 → 更新 `PTE`（恢复 `W`、抹掉 `COW`）→ `kfree` 旧页（内部计数减一：还有共享者就留着，没有就释放）。OOM 时返回 -1，由 `usertrap` 杀死进程，缺页处理器不自己 `panic`。
5. **copyout 防"写穿"**。内核写用户页（如 `read()` 填 `fork` 后的共享缓冲区）走物理地址直写，绕过用户 `PTE` 的只读保护，会把 `COW` 共享页直接写穿。`copyout` 必须逐页先查 `COW` 标记、有则主动调用 `cow_page_fault` 复制后再写。`cowtest` 的 `file` 测试专为此设计：子进程 `read` 进共享 `buf` 后，父进程的 `buf[0]` 必须还是原值。
6. **锁序**。`cow_ref.lock` 与 `kmem.lock` 从不嵌套持有（`kfree` 先释放 `cow_ref` 再取 `kmem`；`kalloc` 反之），两把锁独立成序，无死锁环。

实验内容

1. **COW 标志位**：借 RISC-V 的 `RSW` 保留位（bit 8）：

```
#define PTE_COW (1L << 8) // 自定义的 COW 标志位
```

2. **引用计数**：在 `kernel/kalloc.c` 中建立全局计数数组：

```
struct {  
    struct spinlock lock;  
    int count[PHYSTOP / PGSIZE]; // 每个物理页对应一个引用计数  
} cow_ref;
```

`kinit` 先把所有页计数置 1，再让 `freerange` → `kfree` 逐个减到 0 入 `freelist`；`kalloc` 分配新页时计数重置为 1；`kfree` 改为先减计数，计数大于 0 就直接返回（还有进程在用），减到 0 才真正放回 `freelist`，负数则 `panic`；新增 `cow_ref_add()` 供 `fork` 增加计数，并做地址合法性检查。

3. **uvmcopy：不复制，只共享：**

```

int
uvmcopy(pagetable_t old, pagetable_t new, uint64 sz)
{
    pte_t *pte;
    uint64 pa, i;
    uint flags;
    //char *mem;

    for(i = 0; i < sz; i += PGSIZE){
        if((pte = walk(old, i, 0)) == 0)
            panic("uvmcopy: pte should exist");
        if((*pte & PTE_V) == 0)
            panic("uvmcopy: page not present");
        pa = PTE2PA(*pte);
        flags = PTE_FLAGS(*pte);
        //if((mem = kalloc()) == 0)
            //goto err;
        //memmove(mem, (char*)pa, PGSIZE);
        // 如果页面可写，则剥夺写权限，打上 COW 标记
        if(flags & PTE_W) {
            flags = (flags | PTE_COW) & ~PTE_W;
            *pte = PA2PTE(pa) | flags; // 更新父进程的 PTE
        }
        if(mappages(new, i, PGSIZE, pa, flags) != 0){
            //kfree(mem);直接将相同的物理页映射给子进程，而非 kalloc 分配新页
            goto err;
        }
        // 引用计数加 1
        cow_ref_add((void*)pa);
    }
    return 0;

err:
    uvmunmap(new, 0, i / PGSIZE, 1);
    return -1;
}

```

注意父进程的 PTE 是原地修改的：剥夺 PTE_W、加上 PTE_COW。从这一刻起，父子双方写这个页都会触发缺页。

4. 缺页处理： usertrap 中新增 scause 15（写缺页）分支，交给 cow_page_fault：

```

// [新增] 处理 COW 缺页，返回 0 代表成功，-1 失败
int
cow_page_fault(pagetable_t pagetable, uint64 va)
{
    if(va >= MAXVA) return -1;
    va = PGROUNDOWN(va);

    pte_t *pte = walk(pagetable, va, 0);
    if(pte == 0 || (*pte & PTE_V) == 0 || (*pte & PTE_U) == 0)
        return -1;

    // 必须是在我们 uvmcopy 标记好的 COW 页
    if((*pte & PTE_COW) == 0)
        return -1;

    uint64 old_pa = PTE2PA(*pte);

    // 申请一个新的物理页
    char *new_mem = kalloc();
    if(new_mem == 0) return -1; // 没有空闲内存，进程应被杀死

    // 将旧页的数据复制到新页
    memmove(new_mem, (char*)old_pa, PGSIZE);

    // 更新该虚拟地址的 PTE，指向新页，恢复写权限，抹去 COW 标记
    uint flags = PTE_FLAGS(*pte);
    flags = (flags & ~PTE_COW) | PTE_W;
    *pte = PA2PTE(new_mem) | flags;

    // 释放原先的旧页（其内部会做引用计数减一处理）
    kfree((void*)old_pa);

    return 0;
}

```

四重检查（MAXVA、PTE_V、PTE_U、PTE_COW）层层把关：不是 COW 页的写缺页一律拒绝。
`kfree(old_pa)` 内部计数减一——如果对方进程还共享着，页安然无恙；如果是最后一个引用，

页真正释放。

5. copyout 防"写穿"：内核往用户页拷贝数据走物理地址直写，会绕过用户 PTE 的只读保护。
copyout 在写之前检查 COW 标记并主动触发复制：

```
// 检查目标地址是不是 COW 页面
if(va0 >= MAXVA) return -1;
pte = walk(pagetable, va0, 0);
if(pte != 0 && (*pte & PTE_V) && (*pte & PTE_U)) {
    if(*pte & PTE_COW) {
        // 如果是，主动触发内存分配与替换
        if(cow_page_fault(pagetable, va0) < 0)
            return -1;
    }
} else {
    return -1;
}
```

遇到的问题与解决

kfree 的时序。第一版把引用计数递减放在垃圾填充（`memset(pa,1,PGSIZE)`）之后，共享页被填满 `0x01`，另一个进程读到的数据全坏。计数检查必须在任何破坏性操作之前完成：计数大于 0 立刻返回，页内容原封不动。共享资源的破坏性操作要放在"确认独占"之后。

子进程退出后页未释放。调试时发现 fork 后子进程 `exit`，页仍留在内存里。原因：父子各持一个引用，子进程退出减到 1，父进程还在用，不释放是正确行为。只有双方都退出、计数归零，页才回到 freelist。"最后一个引用消失才释放"是引用计数的核心语义。

实验心得

COW 把 fork 从"复制全部内存"变成"只复制一个页表"，代价是以后每次写共享页时的一次缺页。对 fork+exec 这种常见模式来说很划算，因为大部分页面根本不会被写。

Lab7 : Multithreading

本实验与多线程有关：在用户级线程包中实现上下文切换，用 pthread 修复一个线程不安全的哈希表，并实现线程屏障。

Uthread: switching between threads

实验目的

补全 `user/uthread.c` 与 `user/uthread_switch.S`，实现用户级线程的创建与上下文切换：
`thread_create` 初始化线程，让它第一次被调度时在独立栈上从 `func` 开始执行；
`thread_schedule` 找到下一个 RUNNABLE 线程并调用 `thread_switch`；汇编函数保存当前线程的寄存器、恢复目标线程的寄存器。通过 `uthread` 测试（三个线程各打印 100 轮）。这是第一次手写 RISC-V 汇编，注意汇编里的偏移量和 C 结构体成员要一一对应，错一个字节就是寄存器串位。

设计思路

- 上下文结构。** 仿照内核 `struct context`（`kernel/proc.h`）定义 14 个 `uint64`：`ra`、`sp`、`s0s11`。这 14 个寄存器的选择依据：`ra` 决定"回到哪里"，`sp` 决定"栈在哪里"，`s0s11` 是 callee-saved 寄存器（被调函数承诺保存），其余寄存器（`a0a7`、`t0t6` 等）是 caller-saved，由调用方负责——`thread_switch` 的调用者（C 代码）已经把它们的价值压在自己的栈上了。
- "伪造现场"创建线程。** `thread_create` 不直接执行 `func`，而是伪造一个被切换出去的现场：`context.ra = (uint64)func`（"返回地址"指向函数入口）、`context.sp = (uint64)t->stack + STACK_SIZE`（栈顶，因为栈向下生长）。首次调度时 `thread_switch` 恢复这些寄存器并 `ret`，CPU"返回"到 `ra` 指向的 `func`，线程就在自己的栈上运行。创建线程等于伪造现场，这是本实验的核心设计。
- 汇编切换。** 入参约定（`a0` = 旧 context，`a1` = 新 context）与内核 `swtch.S` 同构，保存 14 个寄存器（`sd ra, 0(a0)` 到 `sd s11, 104(a0)`，偏移 8 字节递增）再对称恢复，最后 `ret`。`thread_schedule` 调用前要先更新 `current_thread` 再传参，新线程醒来时看到的全局状态必须一致。
- 调度策略。** 沿用骨架，从 `current_thread+1` 起线性扫描 RUNNABLE 线程（轮转）。三个测试线程各自启动后先 `yield`，形成确定的 `a→b→c` 启动顺序与 `c→a→b` 轮转节奏。

实验内容

- 线程上下文：**

```
// 用于保存线程执行状态的寄存器上下文
struct context {
    uint64 ra;
    uint64 sp;

    // callee-saved
    uint64 s0;
    ...
    uint64 s11;
};
```

2. **thread_create**: 找到 FREE 槽位，置为 RUNNABLE，并初始化"首次被调度"的现场：

```
// 模拟线程首次被调度时的状态
// 返回地址设为目标函数的入口
t->context.ra = (uint64)func;
// 栈指针设为分配给该线程的独立栈顶（因为栈是向下生长的，所以要加 STACK_SIZE）
t->context.sp = (uint64)t->stack + STACK_SIZE;
```

妙处在于：`thread_switch` 恢复上下文后执行 `ret`，正好"返回"到 `ra` 指向的 `func` —— 创建一个线程，本质是伪造一个被切换出去的现场。

3. **thread_switch 汇编**: 保存当前线程的 14 个 callee-saved 寄存器到旧 context (a0)，从新 context (a1) 恢复：

```

thread_switch:
    /* 保存当前线程的 callee-saved 寄存器到 a0 指向的 struct context
    */

    sd ra, 0(a0)
    sd sp, 8(a0)
    sd s0, 16(a0)
    ...
    sd s11, 104(a0)

    /* 从 a1 指向的 struct context 中恢复目标线程的寄存器 */
    ld ra, 0(a1)
    ld sp, 8(a1)
    ...
    ld s11, 104(a1)
    ret    /* return to ra */

```

遇到的问题与解决

只保存 callee-saved 寄存器的原因（本实验思考题）：函数调用约定中，`thread_switch` 是被 C 代码调用的函数，caller-saved 寄存器（`a0`~`a7`、`t0`~`t6` 等）按约定由调用方负责保存——它们已经躺在调用方的栈上，切换回来自然能恢复。只有 callee-saved 寄存器由被调函数

（`thread_switch`）承诺保存，所以上下文结构里必须显式记录。理解了 ABI，就理解了内核 `swtch` 保存相同寄存器的原因。

实验心得

用户级线程切换和内核进程切换用的是同一套寄存器机制，只是没有特权级转换。线程调度说穿了就是"保存我、恢复你、ret 一下"。

Using threads（ph：哈希表并行化）

实验目的

`notxv6/ph.c` 里的哈希表单线程（`./ph 1`）下零丢失，双线程（`./ph 2`）时会丢失键。原因：两个线程同时 `put` 到同一个 bucket，都读到相同的链表头，各自头插新节点，后写者把先写者的 entry 覆盖（丢失更新）。任务：加锁让 `./ph 2` 零丢失（`ph_safe`），并让双线程吞吐不低于单线程的 1.25 倍（`ph_fast`）。这个实验在宿主机上跑，用 `pthread` 的互斥锁。

设计思路

1. **诊断竞态。** 丢键的根源在 `insert` 的头插法：读 `table[i]` 作为新节点的 `next`，再写 `table[i] = 新节点`。两个线程交错执行时，后写者基于过期的链表头插入，先写者的节点从链表中消失。这是典型的 check-then-act 竞态。
2. **加锁方案。** 一把全局大锁使所有 `put` 串行化，并行加速归零，`ph_fast` 过不了。观察哈希表的访问模式：只有同一 bucket 的链表操作互相干扰，不同 bucket 互不相干。因此为每个 bucket 配一把互斥锁，`put` 时锁 `locks[key % NBUCKET]`，临界区只覆盖链表查找与插入；`get` 是纯读操作（本实验场景下无并发写者），不加锁。
3. **初始化。** `main` 开头循环 `pthread_mutex_init(&locks[i], NULL)`。忘记初始化就加锁是未定义行为，`pthread` 最常见的错误。

实验内容

1. **丢键的原因。** `put` 对同一 bucket 的链表做"头插法"插入：两个线程同时读到相同的链表头，各自插入新节点，后写者覆盖先写者——先插入的 entry 彻底丢失。
2. **每桶一把锁：**

```
struct entry *table[NBUCKET];
// 为每个桶定义一个互斥锁
pthread_mutex_t locks[NBUCKET];

static void put(int key, int value)
{
    int i = key % NBUCKET;

    // 锁定当前需要操作的哈希桶
    pthread_mutex_lock(&locks[i]);
    ... 查找与插入 ...
    pthread_mutex_unlock(&locks[i]);
}
```

`get` 是纯读操作，不加锁（本实验的并发场景下安全）。`main` 开头记得 `pthread_mutex_init` 初始化所有锁——忘记初始化就加锁是未定义行为。

实验心得

并行编程的核心矛盾是正确性与性能的权衡：粗粒度锁正确但慢，细粒度锁快但容易错。答案取决于对共享数据结构的理解——哪些数据真的共享、临界区能缩多小。

Barrier

实验目的

实现线程屏障：所有参与线程都必须到达 `barrier()` 后才能一起继续。`notxv6/barrier.c` 提供了有问题的版本（线程会提前闯关导致断言失败），补全 `barrier()` 使 `./barrier 2` 稳定输出 `OK; passed`。屏障要支持连续多轮：每轮独立计数，轮次记录在 `bstate.round`；还要处理"快线程闯下一轮"的竞态——一个线程刚离开屏障就抢跑回循环，重新进入 `barrier` 把 `bstate.nthread` 又加 1，而上一轮的收尾还在进行，计数错乱导致全员卡死。

设计思路

- 计数器 + 广播。** 所有线程在锁内把 `bstate.nthread` 加 1；最后一个到达的线程（`nthread == 总线程数`）负责 `round++`、`nthread = 0`、`broadcast`；其余线程 `pthread_cond_wait` 挂起。被唤醒的线程重新持锁返回，与最后一个线程一起走出 `barrier`。
- 状态更新的顺序。** 必须先更新 `round`、清零 `nthread`，再 `broadcast`。若顺序颠倒：被唤醒的快线程抢先进入下一轮把 `nthread` 加到 1，此时收尾线程才清零，把下一轮的计数也清掉了，屏障从此"少一个人"，集体卡死。
- 轮次复用。** `nthread` 计数器每轮清零复用，`round` 单调递增记录历史，屏障可无限轮次使用。
- 条件变量与锁的配套。** 检查条件（`nthread` 是否到齐）与挂起等待之间必须原子——否则竞态窗口：线程 A 检查"没到齐"准备睡觉，此刻线程 B（最后一个）到齐并 `broadcast`，然后 A 才睡下——唤醒信号丢失，A 永远等不到下一轮。`cond_wait` 原子释放锁+挂起的语义恰好堵死这个窗口。

实验内容

```

static void
barrier()
{
    // 1. 获取屏障锁，保证对共享状态（nthread, round）的修改是互斥的
    pthread_mutex_lock(&bstate.barrier_mutex);

    // 2. 当前抵达屏障的线程数 + 1
    bstate.nthread++;

    // 3. 判断是不是最后一个抵达的线程
    if (bstate.nthread == nthread) {
        // 如果是，开启下一回合
        bstate.round++;
        // 将计数器清零，留给下一回合使用
        bstate.nthread = 0;
        // [关键] 唤醒所有正在等待该条件变量的线程
        pthread_cond_broadcast(&bstate.barrier_cond);
    } else {
        // 如果不是最后一个线程，则挂起自身，释放互斥锁，进入等待队列
        // 当被 broadcast 唤醒时，会自动重新获取互斥锁往下执行
        pthread_cond_wait(&bstate.barrier_cond, &bstate.barrier_mutex);
    }

    // 4. 所有线程都会执行到这里（无论是最后一个线程，还是被唤醒的等待线程），释放锁
    pthread_mutex_unlock(&bstate.barrier_mutex);
}

```

遇到的问题与解决

共享状态的更新顺序。必须先更新 `round`、清零 `nthread`，然后才 `broadcast`。如果反过来，被唤醒的线程抢先跑到下一轮把 `nthread` 又加回 1，最后一个线程此时才清零，计数被错误清掉，所有线程卡死。

条件变量与互斥锁的配套。 `pthread_cond_wait` 原子地释放锁并挂起，唤醒时再重新获取。如果等待与释放不是原子的，检查条件与进入睡眠之间会出现竞态窗口，另一个线程的 `broadcast` 可能正好落在窗口里，唤醒信号丢失，等待者永远睡下去。

实验心得

屏障是所有并程序的基础积木：矩阵计算的分阶段同步、并行算法的回合制推进都离不开它。"条件变量 + 互斥锁"的背后有大量精密的时序考量，多线程编程的细节决定正确性。

Lab8 : Locks

多核时代，锁竞争是性能的头号杀手。本实验重构 xv6 的两大争用热点——内存分配器和缓冲区缓存——用分而治之的思路削减锁竞争。

Memory allocator

实验目的

重构内存分配器，消除锁竞争。原来的 `kalloc/kfree` 用一把全局锁保护一个全局空闲链表，`kalloc` 里 `kmem` 锁的 `fetch-and-add`（自旋失败次数）达到几万次，是多核下最严重的争用点。任务：给每个 CPU 一个空闲链表、一把锁，不同 CPU 的分配/释放并行；本 CPU 链表空了就从其他 CPU 偷一页。锁名都要以 `kmem` 开头。验收：`kalloc` 的 `test1` 要求 `kmem` 锁的 `fetch-and-add` 总和归零；`test2` 校验页数守恒；`usertests sbrkmuch` 和完整 `usertests` 通过。

设计思路

- 数据结构数组化。** `struct { spinlock lock; struct run *freelist; }` `kmem[NCPU]`。启动时 `freerange` 把全部空闲页放进启动 CPU（CPU 0）的链表，其他 CPU 初始为空，通过分配-释放-偷取自然扩散。
- cpuid 的使用纪律。** `kfree` 把页放回当前 CPU 的链表，`kalloc` 先取当前 CPU 的页。获取 CPU 号前必须 `push_off()` 关中断：中断可能引发调度，进程被挪到别的 CPU，此时拿旧 CPU 号操作链表，两个 CPU 会并发访问同一链表，锁形同虚设。
- 偷取策略。** 本 CPU 链表为空时顺序遍历其他 CPU，偷到一页即止。偷一页而非偷一半：偷取只是兜底，负载均衡交给自然的分配/释放流动；偷一页持目标锁的时间极短。
- 锁序与死锁分析。** `kalloc` 偷取时最多同时持两把锁（本 CPU + 一个目标 CPU），且跳过自己；`kfree` 只取单锁。全系统不存在"持 A 等 B 同时持 B 等 A"的锁序环——唯一理论风险是多 CPU 链表同时为空且互相偷取的 ABBA 窗口，需要所有 CPU 同时耗尽空闲页且恰好成环等待，实验环境下不会触发。
- 页内容消毒保留。** `kfree` 填充 0x01、`kalloc` 填充 0x05 的 `junk` 填充机制原样保留，用于捕获使用已释放内存的悬垂引用。

实验内容

- 改造数据结构：**单一 `kmem` 结构体变成 `NCPU` 元数组：

```
// 将单一结构体改为数组，每个 CPU 一个
struct {
    struct spinlock lock;
    struct run *freelist;
} kmem[NCPU];
```

2. **kinit** 循环初始化每把锁； **freerange** 把所有空闲页放入启动 CPU 的链表。

3. **kfree** 把页放回当前 CPU 的链表：

```
// 将空闲页放回当前 CPU 的 freelist
push_off(); // 关中断，保证 cpuid() 返回值的稳定性
int id = cpuid();
acquire(&kmem[id].lock);

r->next = kmem[id].freelist;
kmem[id].freelist = r;

release(&kmem[id].lock);
pop_off(); // 恢复中断
```

4. **kalloc** 先取本 CPU 的页；空了就遍历其他 CPU，偷到一页即止：


```

// 从当前 CPU 的 freelist 中分配一个空闲页
push_off(); // 关中断, 保证 cpuid() 返回值的稳定性
int id = cpuid();
acquire(&kmem[id].lock);
r = kmem[id].freelist;
if(r)
    kmem[id].freelist = r->next;
else {
    // 当前 CPU 没内存了, 去窃取别的 CPU 的内存
    for(int i = 0; i < NCPU; i++) {
        if (i == id) continue; // 跳过自己

        acquire(&kmem[i].lock);
        if(kmem[i].freelist) {
            r = kmem[i].freelist;
            kmem[i].freelist = r->next;
            release(&kmem[i].lock);
            break; // 偷到一个就够了, 退出循环
        }
        release(&kmem[i].lock);
    }
}
release(&kmem[id].lock);
pop_off(); // 恢复中断

```

改造前 kmem 锁的 fetch-and-add 数以万计、稳居争用榜第一；改造后全部归零。test2 验证页数守恒：反复分配释放后总空闲页数一个不差，偷取机制没有泄漏。

遇到的问题与解决

push_off 的必要性。 最初直接调 cpuid()，测试偶发崩溃。xv6 的 cpuid() 注释明确要求只在关中断时调用：中断可能导致进程被调度到其他 CPU，若拿着旧 CPU 号去操作链表，两个 CPU 会并发访问同一链表，锁形同虚设。用 push_off()/pop_off() 包裹后问题消失。

实验心得

"每 CPU 一个资源池"是消除锁竞争的常用做法，Linux 的 SLUB 分配器也是这个思路。多核优化很多时候就一句话：能不共享就不共享。

Buffer cache

实验目的

重构缓冲区缓存。原来的 bcache 用一把全局锁保护一个 LRU 双向链表，bcachetest 里 fetch-and-add 上万次。和 kalloc 不同，磁盘块是真正共享的资源，没法按 CPU 分。任务：改成哈希表 + 每桶一把锁（桶数 13）；LRU 用时间戳代替链表位置（brelse 时打上 ticks，淘汰时找时间戳最小的块）；未命中时的淘汰过程用一把全局锁串行化。锁名以 bcache 开头。验收：bcachetest 的 test0 要求 bcache 相关锁的 fetch-and-add 总和接近零（小于 500），usertests 通过。不变量：每个磁盘块在缓存中至多一份副本。

设计思路

1. 三层 bget 结构。

- **快速路径**：只锁目标桶 `id = blockno % NBUCKET`，桶内命中则 `refcnt++` 返回。不同块的并发查找互不阻塞，这是消除争用的关键；
- **二次校验**：未命中则获取全局 `eviction_lock`，然后重新检查目标桶——等待全局锁期间别的进程可能恰好把该块装入，不复查会重复装入；
- **全局 LRU 扫描**：按 0→12 的固定顺序逐桶扫描（固定顺序加锁，杜绝交叉等待），找 `refcnt == 0` 且 `timestamp` 最小的块；扫描时"边走边换锁"，最多同时持一把桶锁；选出后将其从旧桶摘除、插入目标桶、更新 `dev/blockno/refcnt`。全局锁全程持有，保证同一时刻只有一个进程在淘汰。

2. 时间戳 LRU。struct buf 新增 timestamp 字段，brelse 在引用计数归零时打上 ticks。链表位置不再代表新旧，brelse 只需锁自己所在的桶。

3. 锁序全局一致。eviction_lock → 桶锁，快速路径只取单桶锁，brelse/bpin/bunpin 只取单桶锁，无人持桶锁再等全局锁，无环无死锁。

4. 不变量维护。摘除旧块、插入新桶、更新标识全程在锁内完成；摘出到插入之间的游离状态受全局锁保护，无人可见——每个块任意时刻至多一份副本、至多在一个桶里。

5. 初始化。所有缓冲块初始全塞进 0 号桶（之后随使用自然散开），每桶哨兵头节点自环初始化，13 把桶锁逐一 initlock。

实验内容

1. **数据结构**：13 个质数桶，每桶一把自旋锁 + 哨兵头节点；另加一把全局 `eviction_lock` 串行化淘汰；struct buf 新增 `timestamp` 字段：

```
// 质数哈希桶数量
#define NBUCKET 13

struct {
    struct spinlock eviction_lock; // 全局锁：用于串行化缓存未命中时的淘汰过程
    struct spinlock lock[NBUCKET]; // 每个哈希桶独立的一把锁
    struct buf buf[NBUF];
    struct buf head[NBUCKET];      // 哈希桶链表数组
} bcache;
```

2. **bget 三层结构**：快速路径（单桶锁命中）→ 二次校验（持全局锁后复查目标桶）→ 全局 LRU 扫描：

```
// 3. 按照 0 到 NBUCKET-1 的顺序遍历寻找全局 LRU（按顺序加锁可绝对避免死锁）
for(int i = 0; i < NBUCKET; i++){
    acquire(&bcache.lock[i]);
    int found_new_min = 0;

    for(b = bcache.head[i].next; b != &bcache.head[i]; b = b->next){
        // 引用计数为 0 代表空闲
        if(b->refcnt == 0 && b->timestamp < min_ticks){
            min_ticks = b->timestamp;
            lru_buf = b;
            found_new_min = 1;
        }
    }
    ...
}
```

扫描时最多同时持一把桶锁：找到更小的就释放旧桶锁、保留新桶锁。

3. **brelse 打时间戳**：引用计数归零时 `b->timestamp = ticks`，替代原来的"插入链表头"。链表位置不再代表新旧，时间戳说了算。

改造前 bcache 锁的 fetch-and-add 高达 16142；改造后归零，桶锁各司其职互不争抢，test0/test1 均通过。

遇到的问题与解决

freeing free block panic。 实现哈希桶版 bget 时，在目标桶中找到可重用块后错误地把它从桶中摘除，导致缓存块从系统中"消失"，随后文件系统操作检测到数据块被重复释放，直接 panic。正确做法：找到可重用的空闲块后只是将其"换桶"（从旧桶摘出、插入新桶），块自始至终都在缓存系统中。缓存一致性是文件系统的核心不变量。

写大文件时 balloc: out of blocks panic。 usertests 的 writebig 测试触发了磁盘块耗尽，原因是实验文件系统镜像太小——把 kernel/param.h 中 FSSIZE 从 1000 调到 10000 后解决。测试不过有时不是代码的错，而是环境配置的问题。

测试不可重复运行。 bcachetest 的 test0 判据原来是"开机以来锁获取总次数 < 500"，跑过一次后再跑必然失败。把判据改成增量式（fork 前先取基线 m，最后判断 $n - m < 500$ ），测试可在同一次启动内反复运行。

实验心得

bcache 的改造比 kalloc 难在没法按 CPU 划分，改用哈希分桶加细粒度锁。快速路径无争用、慢速路径可串行，这个分层思路在很多并发场景都适用。

Lab9 : File system

本实验为 xv6 文件系统添加两项能力：支持大文件（二级间接块）和符号链接。

Large files

实验目的

扩展文件系统支持大文件。原来的文件上限是 268 块（12 个直接块 + 1 个一级间接块），根源在 inode 的 `addrs[]` 数组只有 13 个槽位。任务：把直接块从 12 个减到 11 个，腾出第 13 个槽位放二级间接块（指向 256 个一级间接块，每个再指向 256 个数据块），最大文件变成 $11 + 256 + 256 \times 256 = 65803$ 块（约 16.9 MB）。磁盘 inode 大小不能变（`addrs[]` 保持 13 个 uint）。

`bmap` 要支持二级间接块的按需分配，`itrunc` 要支持完整释放。`bigfile` 测试要写出 65803 块。

设计思路

- 空间腾挪。** `NDIRECT` 从 12 减到 11，`addrs[NDIRECT+2]` 仍是 13 个槽位（64 字节），磁盘 inode 格式完全不变，老文件系统镜像无需重建。槽位语义变为：前 11 个直接块、第 12 个一级间接块（`addrs[NDIRECT]`）、第 13 个二级间接块（`addrs[NDIRECT+1]`）。`MAXFILE = NDIRECT + NINDIRECT + NINDIRECT*NINDIRECT`。
- bmap 的二次索引。** 逻辑块号 `bn` 减掉直接块与一级间接块的区间后，用两次索引定位数据块：`bn / NINDIRECT` 的商索引二级块中的一级间接块号，`bn % NINDIRECT` 的余数索引一级间接块中的数据块号。两层都按需 `balloc` 分配，与原版 `bmap`"用多少建多少"一致。修改

间接块后必须 `log_write(bp)`：间接块也是磁盘元数据，不记日志，崩溃恢复后会出现数据块丢失的静默损坏。

3. **itrunc 的两重释放。** 外层循环遍历二级块中的 256 个一级间接块号，内层循环遍历其中的数据块号逐个 `bfree`；数据块放完后 `bfree` 一级间接块本身，最后 `bfree` 二级块本身并清零 `addrs[NDIRECT+1]`。释放顺序：先数据、后索引、最后顶层，反了会出现"块已释放但指针还在"的悬垂指针。
4. **磁盘容量核算。** 65803 个数据块 + 256 个一级间接块 + 1 个二级块 + 元数据 $\approx$ 66061 块，骨架预置 `FSSIZE=200000` 足够。

实验内容

1. **修改常量：** `NDIRECT` 从 12 减为 11，`addrs[]` 大小改为 `NDIRECT+2`，第 13 个槽位（`addrs[NDIRECT+1]`）成为二级间接块指针：

```
#define NDIRECT 11 // 修改
#define NINDIRECT (BSIZE / sizeof(uint))
// 重新计算最大文件大小：11 个直接块 + 一级间接块 + 二级间接块
#define MAXFILE (NDIRECT + NINDIRECT + NINDIRECT * NINDIRECT)
```

2. **bmap 二级间接块查找：** 逻辑块号减去直接块和一级间接块后进入二级区间，两次索引定位数据块，块不存在时按需 `ballocc` 分配：

```

// 3. Doubly-indirect blocks
if(bn < NINDIRECT * NINDIRECT){
    // 3.1 载入二级间接块的顶层块（分配如果需要）
    if((addr = ip->addrs[NINDIRECT+1]) == 0)
        ip->addrs[NINDIRECT+1] = addr = balloc(ip->dev);

    // 3.2 找到一级目录的块号
    bp = bread(ip->dev, addr);
    a = (uint*)bp->data;
    if((addr = a[bn / NINDIRECT]) == 0){
        a[bn / NINDIRECT] = addr = balloc(ip->dev);
        log_write(bp);
    }
    brelse(bp);

    // 3.3 根据一级目录的块号，深入找到最终数据块
    bp = bread(ip->dev, addr);
    a = (uint*)bp->data;
    if((addr = a[bn % NINDIRECT]) == 0){
        a[bn % NINDIRECT] = addr = balloc(ip->dev);
        log_write(bp);
    }
    brelse(bp);

    return addr;
}
panic("bmap: out of range");

```

修改中间块时必须调用 `log_write(bp)` ——不写日志就修改磁盘块，系统崩溃后日志与磁盘内容将不一致。

3. **itrunc 二级间接块释放**：两层循环——外层遍历 256 个一级间接块，内层遍历其中的数据块逐个 `bfree`，最后释放一级间接块和顶层块自身：

```
// 释放 Doubly-indirect blocks
if(ip->addrs[NDIRECT+1]){
    struct buf *bp1 = bread(ip->dev, ip->addrs[NDIRECT+1]);
    uint *a1 = (uint*)bp1->data;
    // 遍历每一个一级间接块
    for(i = 0; i < NINDIRECT; i++){
        if(a1[i]){
            struct buf *bp2 = bread(ip->dev, a1[i]);
            uint *a2 = (uint*)bp2->data;
            // 遍历一级间接块里的每一个直接块
            for(j = 0; j < NINDIRECT; j++){
                if(a2[j])
                    bfree(ip->dev, a2[j]);
            }
            brelse(bp2);
            bfree(ip->dev, a1[i]); // 释放这个一级间接块本身
        }
    }
    brelse(bp1);
    bfree(ip->dev, ip->addrs[NDIRECT+1]); // 释放顶层目录块本身
    ip->addrs[NDIRECT+1] = 0;
}
ip->size = 0;
iupdate(ip);
```

遇到的问题与解决

间接块索引的计算。 二级间接块里，逻辑块号除以 `NINDIRECT` 的商定位一级间接块（`a[bn / NINDIRECT]`），余数定位数据块（`a[bn % NINDIRECT]`）。第一次实现时把两者搞反，测试直接 panic。在纸上画出“二级块 → 一级块 → 数据块”的树状图再写代码，索引计算的每一层都要先想清楚。

实验心得

大文件实验让我看清了 inode 多级索引的设计：固定 64 字节的 inode 里，直接块管小文件、间接块管大文件，各取所需。

Symbolic links

实验目的

实现 `symlink(target, path)` 系统调用：在 `path` 处创建指向 `target` 的符号链接。符号链接是一个特殊文件，内容是目标路径字符串；`open` 打开它时自动跟随到目标（链接套链接就继续跟，最多 10 层，成环返回错误）；加 `O_NOFOLLOW` 标志时打开链接本身。`target` 不存在也能创建。`symlinktest` 和 `usertests` 都要通过。

设计思路

1. **数据表示。** 复用现有 inode 机制，新增文件类型 `T_SYMLINK`（`stat.h` 中取值 4）。链接内容（目标路径字符串）存在该 inode 的数据块里，不新增数据结构。
2. **sys_symlink。** `create(path, T_SYMLINK, 0, 0)` 创建 inode（`begin_op/end_op` 日志事务包裹），再 `wrotei` 把 `target` 字符串写入数据块。锁规则：`create` 返回时持有 inode 锁，写入完成后无论成败都要 `iunlockput`。边界细节：字符串写入时把结尾 `\0` 一并写入（`strlen(target)+1` 字节）最稳妥，读回时不补 NUL 会导致 `namei` 扫描越界。
3. **open 跟随逻辑。** `sys_open` 拿到 inode 后检查类型：是符号链接且未指定 `O_NOFOLLOW`，进入循环解包——`readi` 读出目标路径 → `iunlockput` 释放当前 inode → `namei(target)` 解析出新 inode → `ilock` 上锁 → 检查是否仍是链接。深度计数器 `depth >= 10` 即报错——链接成环时跟随必然无限循环，深度上限是最简单的护栏。
4. **inode 锁的交接。** 进入循环时持当前 inode 锁；读出路径后先释放旧 inode、再解析新 inode。顺序反了（先 `namei` 后释放）会同时持有两把 inode 锁，两个进程的链接链交叉跟随时可能死锁。
5. **O_NOFOLLOW 的取值。** `0x800`，与既有标志位按位或使用、互不重叠。

实验内容

1. **接线：**新增文件类型 `T_SYMLINK 4`（`stat.h`）、打开标志 `O_NOFOLLOW 0x800`（`fcntl.h`）、系统调用号 22 及全套注册。
2. **sys_symlink：**


```

// symlink 系统调用
uint64
sys_symlink(void)
{
    char target[MAXPATH], path[MAXPATH];
    struct inode *ip;

    // 获取目标路径和软链接存放路径
    if(argstr(0, target, MAXPATH) < 0 || argstr(1, path, MAXPATH) < 0)
        return -1;

    begin_op(); // 开启事务

    // 在 path 位置创建一个类型为 T_SYMLINK 的新 inode
    if((ip = create(path, T_SYMLINK, 0, 0)) == 0){
        end_op();
        return -1;
    }

    // 将 target 字符串（目标路径）写入 inode 的第一个数据块中
    if(writei(ip, 0, (uint64)target, 0, strlen(target)) !=
strlen(target)){
        iunlockput(ip); // 解锁并减少引用
        end_op();
        return -1;
    }

    iunlockput(ip);
    end_op();

    return 0;
}

```

3. sys_open 跟随链接:

```

// 如果打开的是符号链接，且没有指定 O_NOFOLLOW
if(ip->type == T_SYMLINK && !(omode & O_NOFOLLOW)){
    int depth = 0; // 防止软链接成环（死循环），限制最大深度

    // 循环解包，应对"链接指向链接"的情况
    while(ip->type == T_SYMLINK){
        if(depth >= 10){
            iunlockput(ip);
            end_op();
            return -1;
        }

        char target[MAXPATH];
        // 读出里面存储的目标路径
        if(readi(ip, 0, (uint64)target, 0, MAXPATH) <= 0){
            iunlockput(ip);
            end_op();
            return -1;
        }

        iunlockput(ip); // 放开当前的链接文件

        // 解析目标路径，拿到真正的 inode
        if((ip = namei(target)) == 0){
            end_op();
            return -1;
        }

        ilock(ip); // 重新上锁新的 inode 准备下一轮检查
        depth++;
    }
}

```

锁的交接：进入循环时持当前 inode 锁，读完后 `iunlockput` 释放，`namei` 返回新 inode 后再 `ilock` 上锁，任何时刻都清楚谁持有哪把锁。

遇到的问题与解决

符号链接内容的字符串终止。 写入目标路径只写了 `strlen(target)` 字节，没有写结尾的 `\0`；读回时 `readi` 不会自动补 NUL，`namei` 解析时可能扫过缓冲区边界。测试能过是因为内核栈残留恰好为 0，但这种"靠运气"的正确性不可取，稳妥做法是把 `strlen(target)+1` 字节一并写入（或读回后手动补 `target[ip->size]=0`）。

inode 锁的交接顺序。 跟随链接时要"放开旧 inode、锁住新 inode"。如果先 `namei` 再释放旧锁，两把 inode 锁同时持有，两个进程的链接链交叉跟随时可能死锁。先释放、后获取是唯一正确顺序。

实验心得

符号链接把路径变成数据，一层解引用就多一层灵活（Linux 里 `/usr/bin/python -> python3` 就是它）。但它也带来成环、悬空链接这些新问题，都要在实现里处理。

Lab10 : Mmap

实验目的

添加 `mmap` 和 `munmap` 系统调用，实现内存映射文件。本实验只要求子集语义：`addr` 恒为 0（内核选地址）；`prot` 为 `PROT_READ/PROT_WRITE` 的组合；`flags` 为 `MAP_SHARED`（修改写回文件）或 `MAP_PRIVATE`（不写回）；`offset` 恒为 0。`mmap` 要惰性分配：调用时不读文件、不分配物理页，只记录 VMA 并返回地址；第一次访问时缺页，内核分配物理页、从文件读入一页、建立映射。`munmap` 解除映射，`MAP_SHARED` 的脏页写回文件，`munmap` 可以只解除头部、尾部或整个区域。`fork` 后子进程继承映射，`exit` 时自动 `munmap`。`mmaptest`（含 `fork_test`）和 `usertests` 都要通过。

设计思路

- VMA 数组。** `struct proc` 挂固定大小 16 个的 `struct vma` 数组（xv6 内核没有内核态内存分配器，固定数组是官方认可的简化）。每个 VMA 记录：valid 标志、映射起始地址、长度、权限、标志、文件指针、文件内偏移。
- 地址分配策略。** 从 `MAXVA - 2*PGSIZE`（`TRAPFRAME` 的位置，`trampoline/trapframe` 之下）开始向下堆叠——新映射的上界 = 现有 VMA 的最低地址，再向下减 `PGROUNDUP(length)`。天然页对齐、各映射互不重叠、避开堆（`sz` 以下）与 `trampoline/trapframe` 区域。
- mmap 只记账。** `sys_mmap` 做四件事——权限校验（`fd` 有效且文件可读；`MAP_SHARED + PROT_WRITE` 要求文件可写）、找空 VMA 槽、算地址、填 VMA + `filedup(f)`。`filedup` 是关键：VMA 持有自己的文件引用，用户随后 `close(fd)` 只减用户那一份，文件对象在映射存活期间不销毁。

4. **缺页处理。** `usertrap` 拦截 `sz <= va < MAXVA` 区间的缺页（该区间只可能是 `mmap` 区域），按 `va` 查找所属 VMA → `kalloc` 清零 → `begin_op/iLOCK` 后 `readi` 读入该页对应文件内容 → 按 `prot` 设置 PTE 权限（`PTE_U` + 可选的 `R/W/X`）→ `mappages` 映射。读到 EOF 之后的部分保持全零，符合“映射超出文件处读出 0”的语义。
5. **`munmap` 的脏页回写。** `MAP_SHARED` 时逐页检查 `PTE_D`（Dirty）位——RISC-V 硬件在页被写过时自动置位，只回写真正写过的页（避免把从未写过的只读页写回文件）。写回用 `writei(ip, 1, va, offset, PGSIZE)`（`user_src=1` 表示源是用户虚拟地址）。之后 `uvmunmap` 解除映射（配合放宽版 `uvmunmap`：未映射页跳过），调整 VMA 记录（头部解除时同步平移 `addr/offset`，尾部解除只减 `length`），长度归零时 `fileclose` 并释放槽位。
6. **`fork` 继承与 `exit` 清理。** `fork` 复制 VMA 数组并逐项 `filedup`；物理页不复制（`mmap` 区域在 `sz` 之上，`uvmcopy` 只走 `[0,sz)`），子进程缺页时自行从文件懒加载。`exit` 时 `vma_exit` 遍历所有有效 VMA 逐个 `munmap`，脏页落盘、引用释放、无泄漏。

实验内容

1. **VMA 结构：** `struct proc` 中挂固定大小的 VMA 数组：

```
// 虚拟内存区域结构
#define NVMA 16
struct vma {
    int valid;           // 是否被使用
    uint64 addr;         // 映射的虚拟起始地址
    uint64 length;       // 映射长度
    int prot;            // 权限
    int flags;           // 标志 (SHARED/PRIVATE)
    struct file *vfile;  // 映射的文件指针
    uint64 offset;       // 文件偏移量
};
```

2. **`sys_mmap`：只记账，不干活。** 权限检查（只读文件不允许“`MAP_SHARED + PROT_WRITE`”；`fd` 对应的文件必须可读）；地址从 `MAXVA - 2*PGSIZE` 向下堆叠分配；填 VMA 后 `filedup(f)` 增加文件引用计数，返回映射地址：

```
// 寻找可用的高位虚拟地址（避开 heap，从 MAXVA 向下寻找）
uint64 curr_addr = MAXVA - 2 * PGSIZE; // 留出 trapframe 和
trampoline 的空间
for(int i = 0; i < NVMA; i++) {
    if(p->vma[i].valid && p->vma[i].addr < curr_addr) {
        curr_addr = p->vma[i].addr;
    }
}
```

3. 缺页时才真正干活。 `usertrap` 拦截 `mmap` 区域的缺页（`fault_va >= p->sz && fault_va < MAXVA`），交给 `handle_mmap_page_fault`：

```

// mmap 懒加载处理
int handle_mmap_page_fault(uint64 va)
{
    struct proc *p = myproc();
    struct vma *v = 0;

    // 查找出问题的虚拟地址属于哪个 VMA
    for(int i = 0; i < NVMA; i++) {
        if(p->vma[i].valid && va >= p->vma[i].addr && va < p->vma[i].addr +
p->vma[i].length) {
            v = &p->vma[i];
            break;
        }
    }
    if(v == 0) return -1; // 不属于 mmap 区域

    // 分配新的物理页
    char *mem = kalloc();
    if(mem == 0) return -1;
    memset(mem, 0, PGSIZE);

    // 从文件中读取该页对应的内容
    begin_op();
    ilock(v->vfile->ip);
    uint64 offset = v->offset + PGROUNDDOWN(va) - v->addr;
    readi(v->vfile->ip, 0, (uint64)mem, offset, PGSIZE);
    iunlock(v->vfile->ip);
    end_op();

    // 根据 VMA 设置页表权限
    int flags = PTE_U;
    if(v->prot & PROT_READ) flags |= PTE_R;
    if(v->prot & PROT_WRITE) flags |= PTE_W;
    if(v->prot & PROT_EXEC) flags |= PTE_X;

    // 将物理页映射到发生缺页的虚拟地址
    if(mappages(p->pagetable, PGROUNDDOWN(va), PGSIZE, (uint64)mem, flags)

```

```

!= 0) {
    kfree(mem);
    return -1;
}
return 0;
}

```

物理页预先清零，`readi` 读到文件末尾之后时页面保持全零，符合"映射超出文件处读出 0"的语义。

4. munmap 与脏页回写。解除映射时 MAP_SHARED 的脏页（PTE_D 置位）写回文件，然后解除映射、调整 VMA：

```

// 如果是 SHARED 模式，必须将修改过 (Dirty) 的页面写回磁盘
if(v->flags & MAP_SHARED) {
    for(uint64 a = addr; a < addr + length; a += PGSIZE) {
        pte_t *pte = walk(p->pagetable, a, 0);
        if(pte && (*pte & PTE_V) && (*pte & PTE_D)) { // 检查页面是否有效且被
弄脏过
            begin_op();
            ilock(v->vfile->ip);
            // 写回文件
            writei(v->vfile->ip, 1, a, v->offset + (a - v->addr), PGSIZE);
            iunlock(v->vfile->ip);
            end_op();
            *pte &= ~PTE_D; // 清除脏位
        }
    }
}

// 解除内存映射 (真正的物理页释放)
uvmunmap(p->pagetable, addr, PGROUNDUP(length) / PGSIZE, 1);

```

之后调整 VMA 记录（支持从头部或尾部部分解除，头部解除时同步平移 offset）；VMA 长度归零时 `fileclose` 释放文件引用、槽位置为无效。配合把 `uvmunmap` 放宽为"未映射页跳过"——`munmap` 一个从未触碰过的区域不 panic，对应 `mmaptest` 的 not-mapped unmap 用例。

5. 进程生命周期：fork 时子进程复制父进程的 VMA 数组并逐项 `filedup`（物理页不复制，子进程缺页时自行懒加载）；exit 时 `vma_exit` 遍历所有有效 VMA 逐个 `do_munmap`，MAP_SHARED 的修改完整落盘。

遇到的问题与解决

close(fd) 之后映射仍可用。这是 mmap 语义的经典细节：`filedup` 让 VMA 持有自己的文件引用，用户 close 只减掉用户的一份。文件对象由引用计数守护，映射存活期间文件不会被销毁，这体现了 Unix 文件对象与文件描述符的分离。

只读映射被写入时的行为。缺页处理器不区分读/写缺页，对已映射页再次 `mmap` 会触发 `remap panic`，内核整体崩溃而非只杀肇事进程。测试没有覆盖这个场景，严谨的做法应先 `walk` 检查是否已映射、再按 PTE 权限拒绝非法写入。这是实现中的一处已知薄弱点。

实验心得

mmap 把前面几个实验的知识串起来了：VMA 是进程地址空间的管理结构，懒加载复用 lazy 的缺页处理，脏位回写用上 COW 的 PTE 标志，`filedup` 用到文件系统的引用计数。

Lab11 : Network driver

实验目的

补全 E1000 网卡驱动：实现 `kernel/e1000.c` 里的 `e1000_transmit`（把网络栈送来的帧填进发送环交给网卡发出）和 `e1000_recv`（扫描接收环，把收到的包交给 `net_rx`，并补充新缓冲区）。上层协议栈（IP/UDP/ARP）由骨架提供。测试：`make server` 起宿主机回显服务器，xv6 里跑 `nettests`——ping、100 次单进程 ping、10 进程并发 ping、真实 DNS 查询，全部通过。驱动主要和三个机制打交道：DMA 描述符环、DD 位（硬件置位表示处理完）、中断确认。

设计思路

- 发送：TX 环的入队协议。**发送环是 16 个 `tx_desc` 描述符的循环数组，软件维护尾指针 TDT（下一个可写位置），硬件按序取走。`e1000_transmit` 的完整协议：
 - 先 `acquire(&e1000_lock)`——多个进程可能并发发包，发送环是共享资源；
 - 读 `regs[E1000_TDT]` 拿当前索引，检查该槽位描述符的 DD 位：未置位说明硬件还没发完上一包，环满，返回 -1（由调用方 `net_tx_eth` 释放 mbuf，所有权边界清晰）；
 - DD 已置位：先 `mbuffree(tx_mbufs[idx])` 释放该槽位上一次发送的旧包（硬件已发完，释放安全，提前释放会被 DMA 读飞），再填新描述符：`addr = m->head`、`length = m->len`、`cmd = E1000_TXD_CMD_EOP | E1000_TXD_CMD_RS`（EOP 表示完整包；RS 要求硬件发完后置 DD 位报告状态）；

- 把 mbuf 存进 `tx_mbufs[idx]` 备忘（将来好释放），更新 `regs[E1000_TDT] = (idx+1) % TX_RING_SIZE` 通知硬件，解锁返回 0。
2. **接收：RX 环的收割协议。** 接收环也是 16 槽循环数组，硬件 DMA 进 `rx_mbufs` 缓冲并置 DD 位、发中断；软件维护尾指针 RDT。 `e1000_recv` 的 while 循环：`idx = (RDT+1) % RX_RING_SIZE` 取下一个待收包 → DD 位未置说明没有新包，退出 → 置位则把 mbuf 的 `m->len` 更新为描述符记录的长度，交给 `net_rx(m)`（协议栈负责释放）→ 立刻 `mbufalloc` 分配新缓冲补进该槽位、更新描述符 `addr`、清 `status` → 回写 RDT。每收一包补一包，硬件永远有缓冲可写。
 3. **锁的取舍。** `transmit` 加锁（多进程共享 TX 环），`recv` 不加锁——接收只在中断处理上下文运行（同 CPU 上设备中断已屏蔽，不会与自身并发），且 TX/RX 环是完全独立的数据结构。给 `recv` 加锁不仅多余，还可能死锁：进程持锁发包时被同 CPU 中断打断，中断处理再抢同一把锁。
 4. **中断确认。** 本分支骨架已含官方后续的 "fix interrupt ack" 修复——`e1000_intr` 中先向 ICR 寄存器写 `0xffffffff`（write-1-to-clear，清全部中断位）再调 `e1000_recv`。不确认中断，网卡不再产生新中断，网络就此沉默。
 5. **初始化已由骨架完成。** `e1000_init` 分配 TX/RX 环与 mbuf、预置 TX 描述符 DD=1（开机后 16 个槽位全部可用）、配置 RX 每包一中断等，驱动只需专注收发两个函数。

实验内容

背景：DMA 与描述符环。 E1000 网卡通过 DMA 与内存交互：网卡不经过 CPU，直接从内存中的缓冲区读走待发送的包、把收到的包写进内存。缓冲区的地址存放在描述符环中——TX 环用于发送，RX 环用于接收，都是循环数组。软件维护尾指针（TDT/RDT 寄存器），硬件维护头指针，双方通过描述符中的 DD（Descriptor Done）位沟通：硬件置位 DD 表示"这包处理完了"。

e1000_transmit：入队一个待发送包：

```
int
e1000_transmit(struct mbuf *m)
{
    // 1. 获取 E1000 发送锁，保证并发安全
    acquire(&e1000_lock);

    // 2. 获取当前的发送环索引 (TDT: Transmit Descriptor Tail)
    uint32 idx = regs[E1000_TDT];

    // 3. 检查环形缓冲区是否已满：判断该位置上一次的数据是否已经被硬件发送完毕 (DD 位)
    if((tx_ring[idx].status & E1000_TXD_STAT_DD) == 0) {
        // DD 位没置 1，说明硬件还在忙，环满了
        release(&e1000_lock);
        return -1;
    }

    // 4. 如果这个位置之前残留着上一次发送的 mbuf，将其释放
    if(tx_mbufs[idx]) {
        mbuf_free(tx_mbufs[idx]);
        tx_mbufs[idx] = 0;
    }

    // 5. 将新的 mbuf 的物理地址和长度填入描述符
    tx_ring[idx].addr = (uint64)m->head;
    tx_ring[idx].length = m->len;

    // 设置命令标志位：
    // EOP (End Of Packet)：表示这是一个完整的包
    // RS (Report Status)：告诉网卡在发送完成后，将 status 的 DD 位置为 1
    tx_ring[idx].cmd = E1000_TXD_CMD_EOP | E1000_TXD_CMD_RS;

    // 6. 把新 mbuf 的指针记录在 tx_mbufs 中，防止内存泄漏，下次好释放
    tx_mbufs[idx] = m;

    // 7. 更新 TDT 寄存器，通知网卡硬件开始发送
    regs[E1000_TDT] = (idx + 1) % TX_RING_SIZE;
```

```
// 8. 释放锁
release(&e1000_lock);

return 0;
}
```

每个细节都有讲究：`mbuffree` 旧包必须在 DD 已置位（硬件发完）之后，提前释放会被 DMA 中的网卡读飞；返回 -1 时包由调用方（net.c）释放，所有权边界清楚；加锁是因为多个进程可能并发发包。

e1000_recv：收割到达的包：

```
static void
e1000_recv(void)
{
    // 一次中断可能收到多个包，循环读取直到没有新包为止
    while (1) {
        // 1. 获取下一个要读取的环索引 (RDT 指向最后被消费的，所以要 +1)
        uint32 idx = (regs[E1000_RDT] + 1) % RX_RING_SIZE;

        // 2. 检查该描述符是否有新数据到达 (判断 DD 位)
        if((rx_ring[idx].status & E1000_RXD_STAT_DD) == 0) {
            // 没有新数据了，退出循环
            break;
        }

        // 3. 取出装有新数据的 mbuf
        struct mbuf *m = rx_mbufs[idx];

        // 4. 更新 mbuf 的有效数据长度
        m->len = rx_ring[idx].length;

        // 5. 将数据包上传给网络协议栈 (net_rx 内部会负责处理该包，无需我们释放)
        net_rx(m);

        // 6. 为该槽位重新分配一个新的空白 mbuf，供网卡将来接收新数据使用
        m = mbufalloc(0);
        if (!m) {
            panic("e1000_recv: mbufalloc failed");
        }
        rx_mbufs[idx] = m;

        // 7. 更新描述符，指向新的 mbuf 地址，并清空 status
        rx_ring[idx].addr = (uint64)m->head;
        rx_ring[idx].status = 0;

        // 8. 更新 RDT 寄存器，告诉网卡这个槽位可以再次写入了
        regs[E1000_RDT] = idx;
    }
}
```

```
}  
}
```

关键模式：每收走一个包，立刻补给一个新的空 mbuf，网卡的 DMA 永远有地方写。一次中断可能攒了多个包，while 循环收割到 DD 位不再置位为止。

另外，本分支的骨架已包含官方后续的 "fix interrupt ack" 修复：中断处理函数中先向 ICR 寄存器写 0xffffffff 确认中断，再调用 e1000_recv。

遇到的问题与解决

发送环满的处理。 发送环只有 16 个槽位，硬件处理速度跟不上时环会满。此时 e1000_transmit 返回 -1 丢包，由上层协议兜底（TCP 会重传，UDP 本就不保证可靠）。返回错误而非无限等待，是驱动层最务实的选择。

recv 不加锁的理由。 接收函数只在中断处理上下文执行（同一 CPU 上设备中断已被屏蔽），不会与自身并发；TX 环和 RX 环是独立的数据结构。给 recv 加锁不仅多余，还可能死锁：进程持锁发包时被同 CPU 的中断打断，中断处理再去抢同一把锁。锁的每一处省略，都要有充分的并发模型论证。

实验心得

网卡驱动是操作系统与真实硬件对话的窗口，也是整个课程距离硬件最近的实验。DMA 环、DD 标志、内存屏障、中断确认，每一步都要严格遵循硬件手册的契约。用 QEMU 抓包文件核对 ARP、UDP 报文交换的过程，完整呈现了从应用层到物理层的整条网络路径。